Report

Cloud Computing At AlfidoTech

Submitted

To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Roll Number of Student:25scs1003005429

Name of Student: PALASH MALVIYA

Batch: 2025-2029

2026-27

CANDIDATE'S DECLARATION

I, PALASH MALVIYA do hereby declare that the internship report titled **"Cloud Computing At AlfidoTech"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature
Student Name : PALASH MALVIYA
Roll No: 25SCS1003005429

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I express my gratitude to my parents for their blessings.

Signature
Student Name:PALASH MALVIYA
Roll No:25SCS1003005429

INTERNSHIP COMPLETION CERTIFICATE

Fig 3.1: Internship Completion Certificate issued by Alfido Tech for the 1-Month Cloud Computing Internship (Candidate ID: BS/REG/126233)

TABLE OF CONTENTS

Note: Page references are aligned with the final formatted report layout.

4. PROJECT DESCRIPTION

4.1 Introduction

This report presents the work carried out during the Cloud Computing Internship undertaken at Alfido Tech. The internship was designed to provide hands-on, practical exposure to core cloud computing services offered by Amazon Web Services (AWS), moving beyond theoretical concepts into real deployment scenarios. Over the course of the internship, four independent tasks were completed, each targeting a distinct area of cloud infrastructure: static website hosting, virtual machine provisioning and web server configuration, serverless computing, and containerized application deployment.

The tasks were performed using the AWS Free Tier console and covered services including Amazon S3, Amazon EC2, AWS Lambda, Amazon API Gateway, and Docker running on a cloud virtual machine. Each task was executed end-to-end - from resource provisioning and configuration, through deployment, to verification of the final publicly accessible result - and documented with screenshots captured directly from the AWS Management Console and the deployed applications.

4.2 Organization Profile

Organization Name: Alfido Tech

Alfido Tech is a Hyderabad, Telangana (India) based technology training organization that runs structured, task-based internship programs for engineering students, with a focus on emerging domains such as Cloud Computing. The organization is registered under MSME and ISO 9001:2015, and its internship programs are recognized in line with AICTE guidelines. Alfido Tech’s internship model is built around real-world, hands-on tasks rather than purely theoretical modules, supported by curated learning resources, mentor guidance, and an internship coordination team. On successful completion of all assigned tasks, interns are awarded an official Internship Completion Certificate, and high-performing interns may additionally become eligible for a Letter of Recommendation (LOR).

Supervisor / Mentor Name: Akash Dubey

Internship Duration: 10th August 2026 to 10th September 2026

4.3 Problem Statement

Traditional on-premises infrastructure requires significant upfront investment, manual server management, and lacks the elasticity needed to handle variable workloads. Organizations increasingly need to deploy websites, applications, backend logic, and containerized services in a manner that is scalable, cost-effective, and quickly provisioned. The problem addressed through this internship was to gain practical competence in deploying and managing such services on a public cloud platform (AWS), covering the complete spectrum from static content hosting to compute, serverless execution, and containerization, while ensuring that each deployment is configured securely and is publicly verifiable.

4.4 Project Objectives

- To deploy a static website using Amazon S3 with public access configured through bucket policies.
- To provision a virtual machine on Amazon EC2, configure network security, and install and run a web server (Apache2) on it.
- To design and deploy a serverless function using AWS Lambda and expose it publicly through Amazon API Gateway.
- To deploy a containerized application (Nginx) using Docker on a cloud-hosted Ubuntu virtual machine and make it publicly accessible.
- To document each deployment with configuration details, commands used, and verification evidence (screenshots).

4.5 Scope of the Project

The scope of this internship was limited to four self-contained cloud deployment tasks performed on the AWS Free Tier, within the eu-north-1 (Stockholm) AWS region. The work covered storage-based static hosting (S3), compute provisioning (EC2), serverless function execution with API exposure (Lambda + API Gateway), and containerized deployment (Docker on EC2). The scope did not extend to advanced production concerns such as custom domain mapping, HTTPS/TLS certificates, auto-scaling, load balancing, CI/CD pipelines, or infrastructure-as-code automation; these represent natural extensions for future work.

4.6 Technologies and Tools Used

Category	Technologies / Tools
Cloud Platform	Amazon Web Services (AWS)
Storage / Hosting	Amazon S3 (Static Website Hosting)
Compute	Amazon EC2, Ubuntu Server
Web Server	Apache2
Serverless Computing	AWS Lambda (Python 3.12)
API Management	Amazon API Gateway (REST API)
Containerization	Docker, Nginx
Languages	HTML, CSS, JavaScript, Python
Operating System	Ubuntu Server (24.04 LTS)

4.7 System Architecture

This section presents the architecture of each of the four deployments, showing the flow of a client request through to the deployed resource on AWS.

Task 1 - Static Website on Amazon S3

User → Internet → Amazon S3 (Static Website)

Task 2 - Virtual Machine with Web Server

User → Internet → AWS EC2 (Ubuntu) → Apache2 → Webpage

Task 3 - Serverless Function

Client → API Gateway → AWS Lambda (Python) → JSON Response

Task 4 - Docker Container on Cloud VM

User → Internet → AWS EC2 (Ubuntu) → Docker → Nginx Container → Webpage

4.8 Methodology

The methodology adopted for the internship followed a consistent, task-by-task approach: provisioning the required AWS resource, applying the necessary security/network configuration, installing or deploying the target software, verifying functionality through the AWS console, and finally validating public accessibility through a browser. Each of the four tasks is detailed below.

4.8.1 Task 1: Deploy a Static Website on AWS S3

Objective: To deploy a static website using Amazon S3.

Tools Used: AWS S3, HTML, CSS, JavaScript.

Bucket Name: task1-alfido-static-website

Website URL: http://task1-alfido-static-website.s3-website.eu-north-1.amazonaws.com/

Procedure:

- Created an Amazon S3 bucket named task1-alfido-static-website.
- Configured the bucket for static website hosting.
- Disabled Block Public Access to allow public access to the website.
- Uploaded the website files, including index.html and the required CSS/JavaScript files.

- Configured an S3 bucket policy to allow public s3:GetObject access.
- Set index.html as the index document for the website.
- Obtained the S3 static website endpoint from the bucket Properties section.
- Opened the endpoint in a web browser and verified that the website was successfully deployed and accessible publicly.

S3 Bucket Policy:

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "PublicReadGetObject",
   "Effect": "Allow",
   "Principal": "*",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::task1-alfido-static-website/*"
  }
 ]
}
```

Screenshots / Evidence:

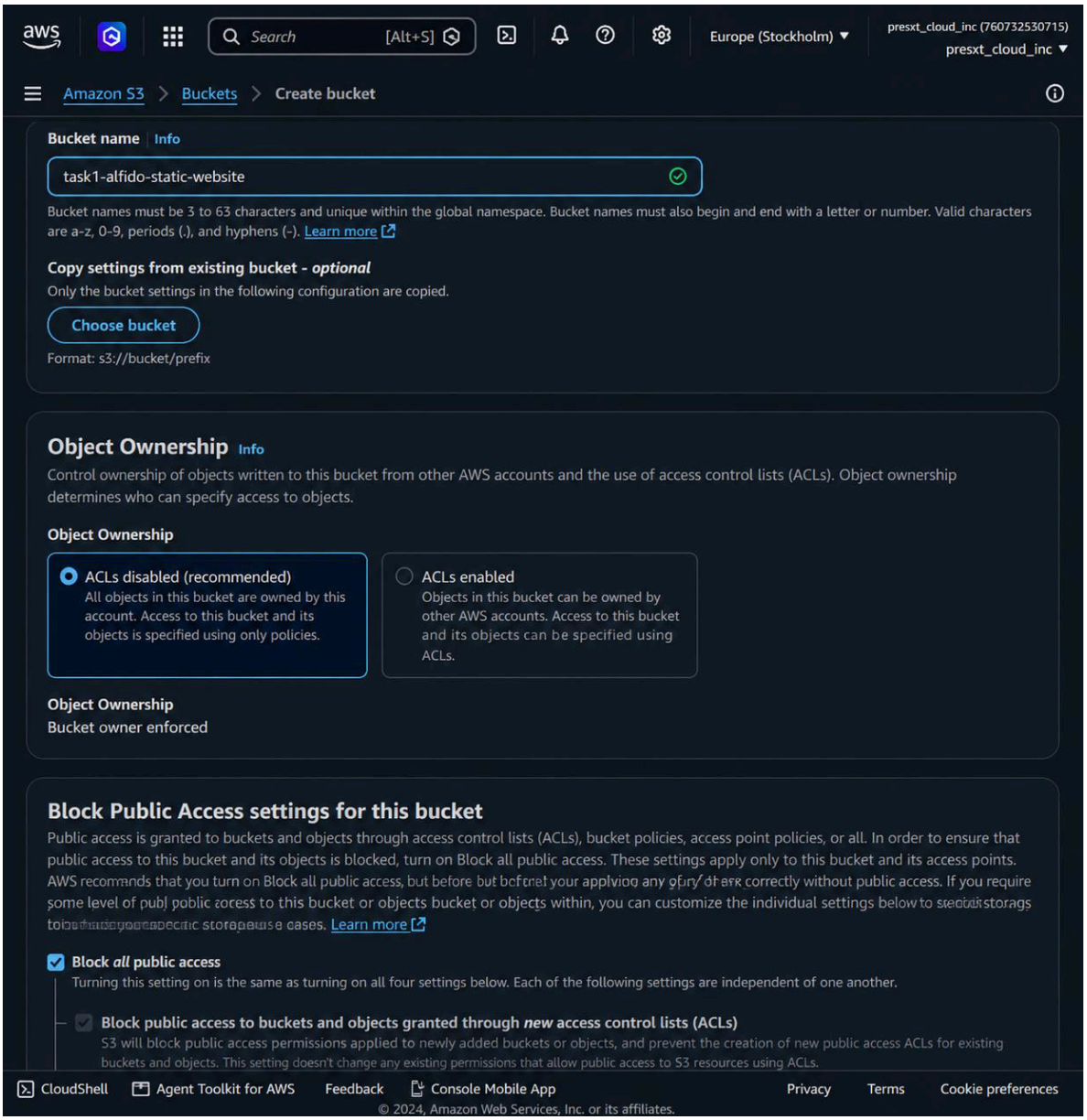

Fig 4.8.1(a): Creating the S3 bucket "task1-alfido-static-website"

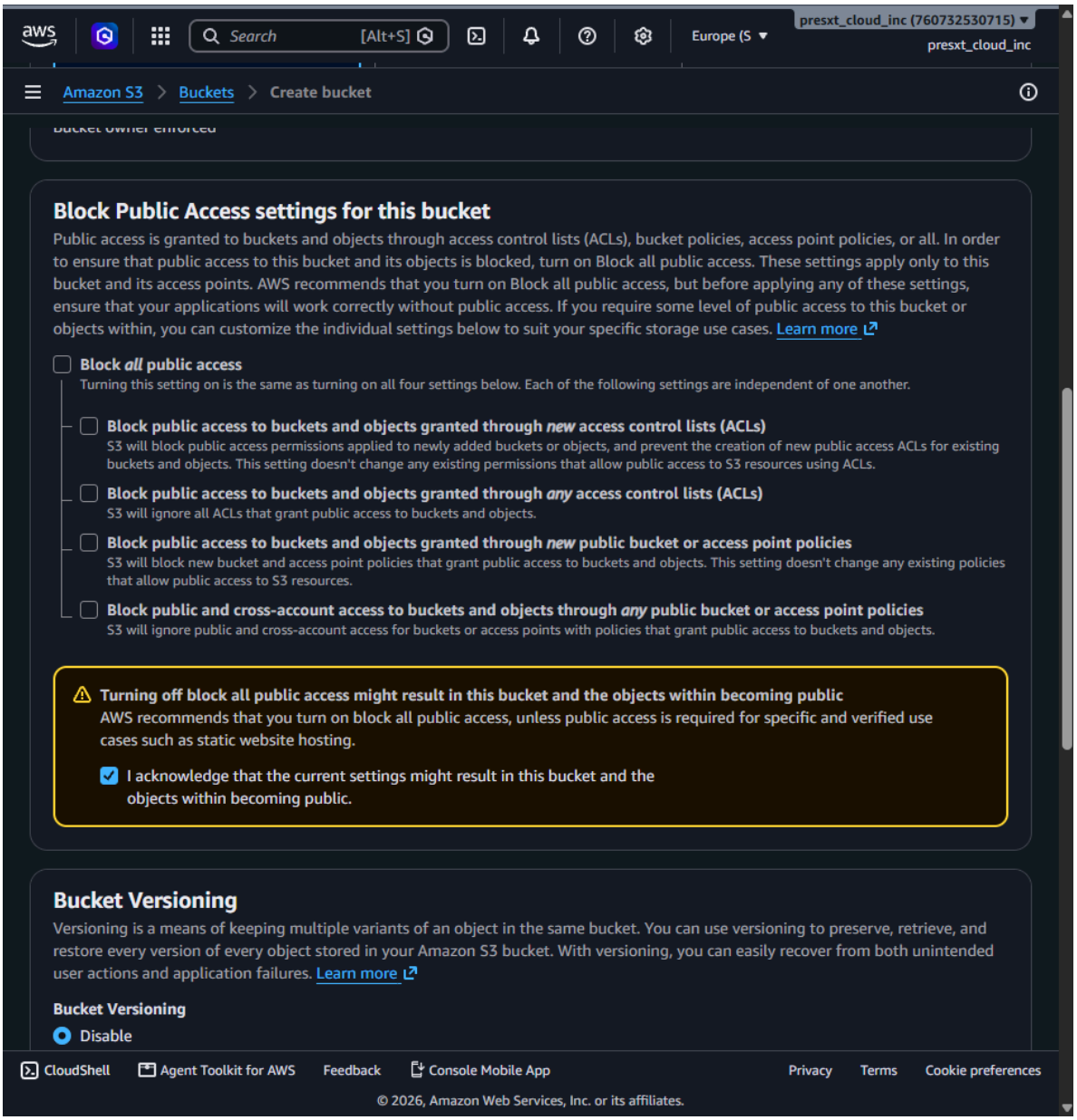

Fig 4.8.1(b): Disabling Block Public Access for the bucket

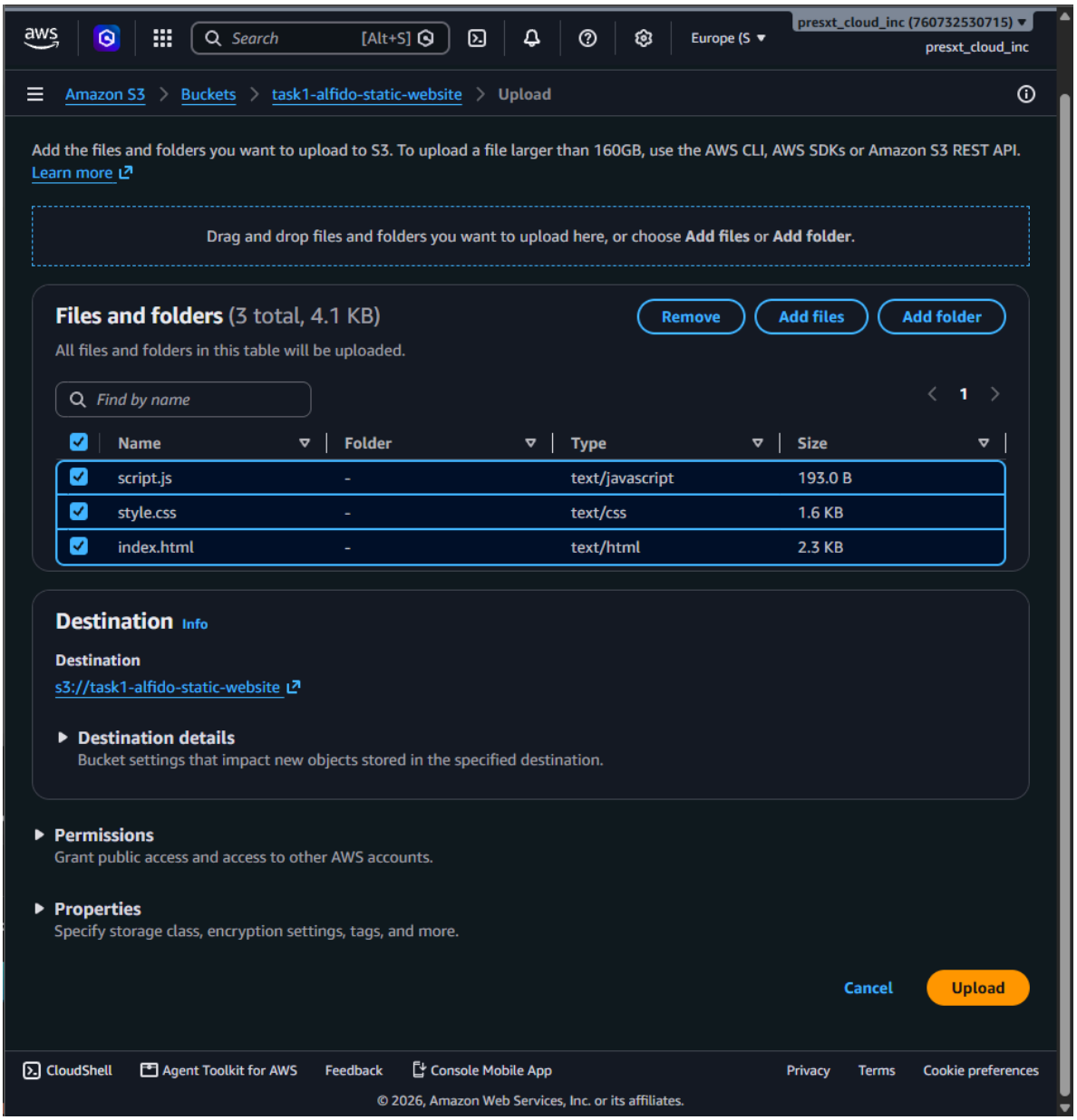

Fig 4.8.1(c): Uploading website files (index.html, style.css, script.js) to the bucket

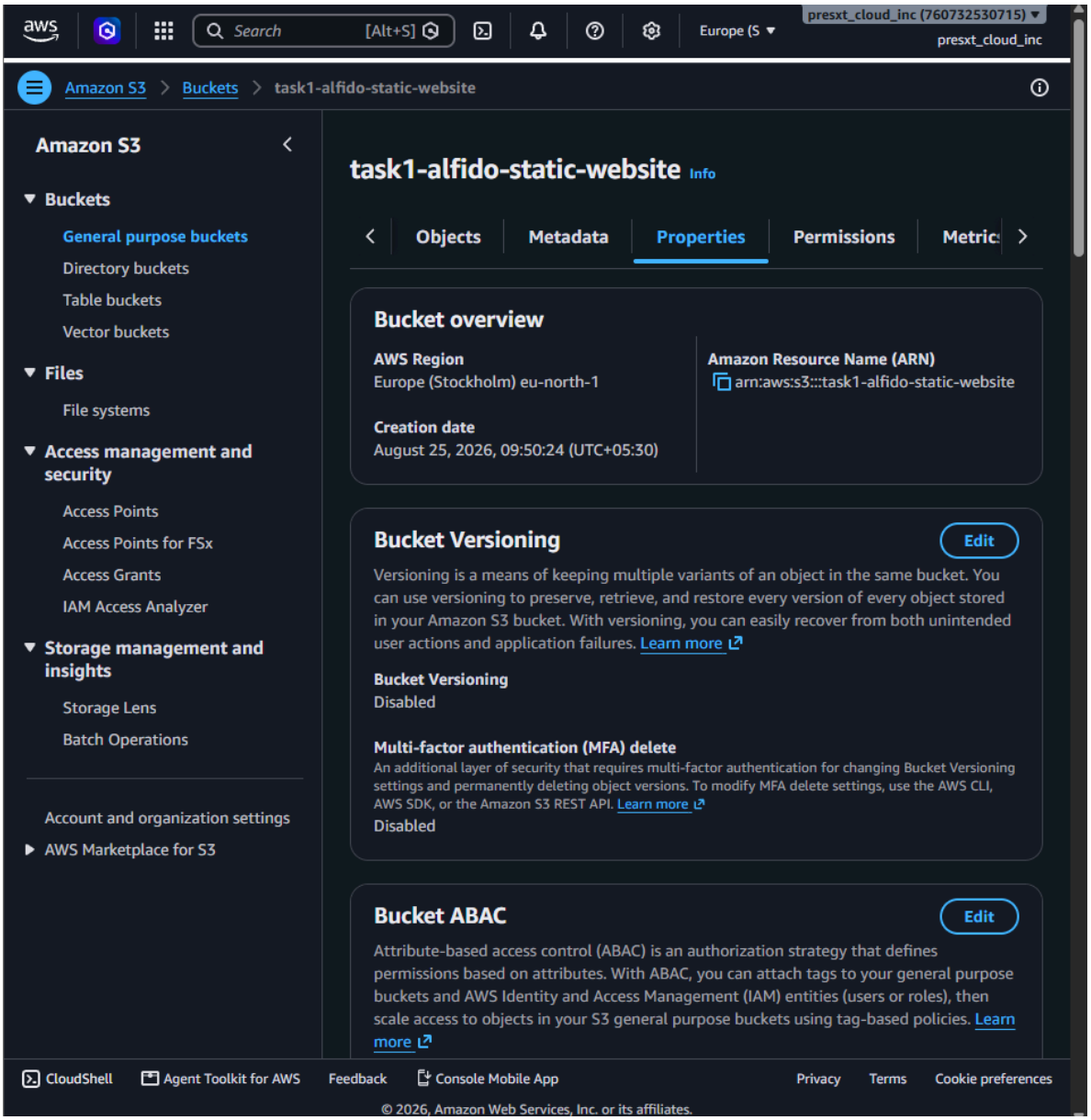

Fig 4.8.1(d): Bucket overview showing region and configuration details

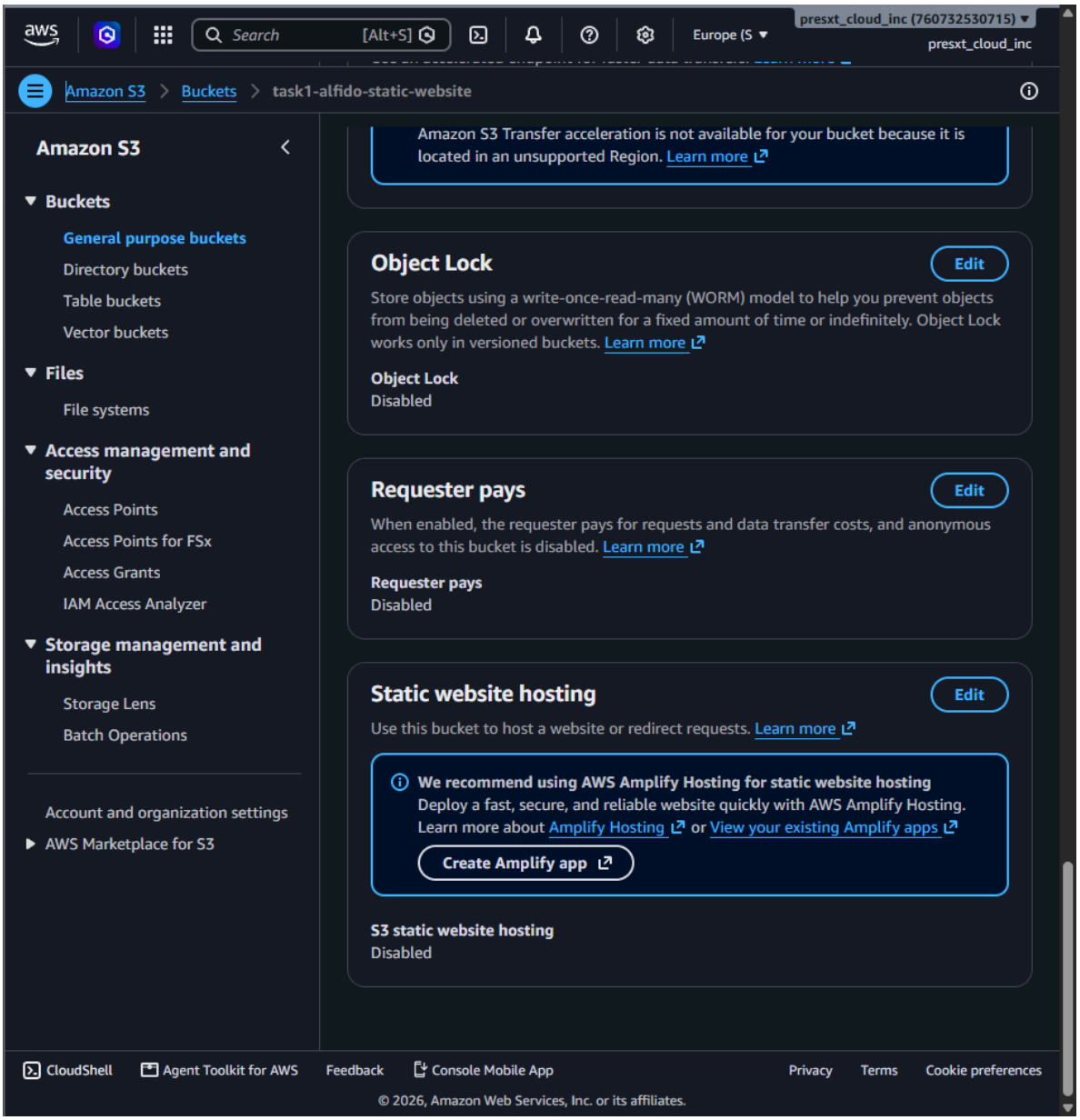

Fig 4.8.1(e): Static website hosting section before enabling hosting

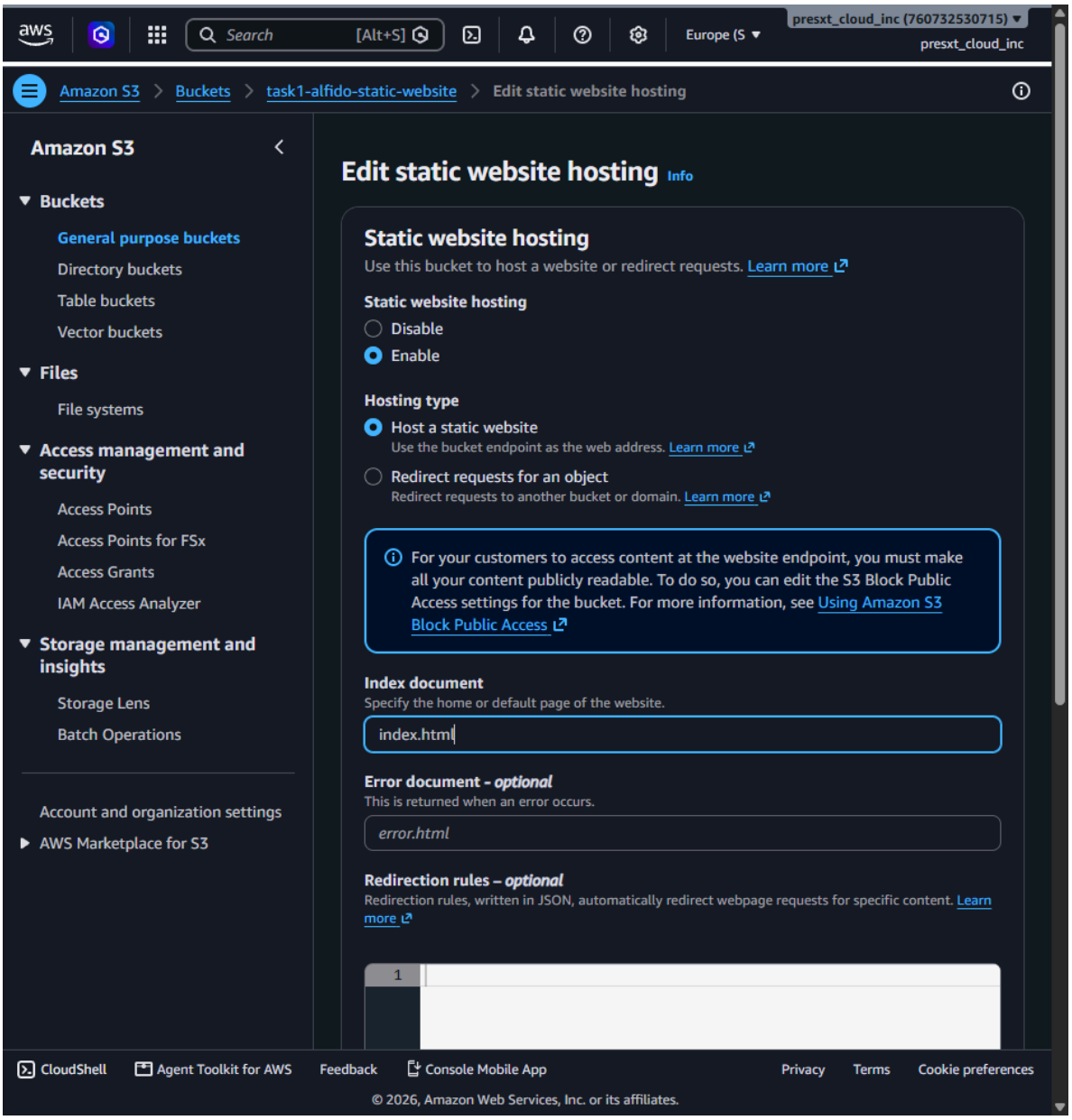

Fig 4.8.1(f): Enabling static website hosting with index document set to index.html

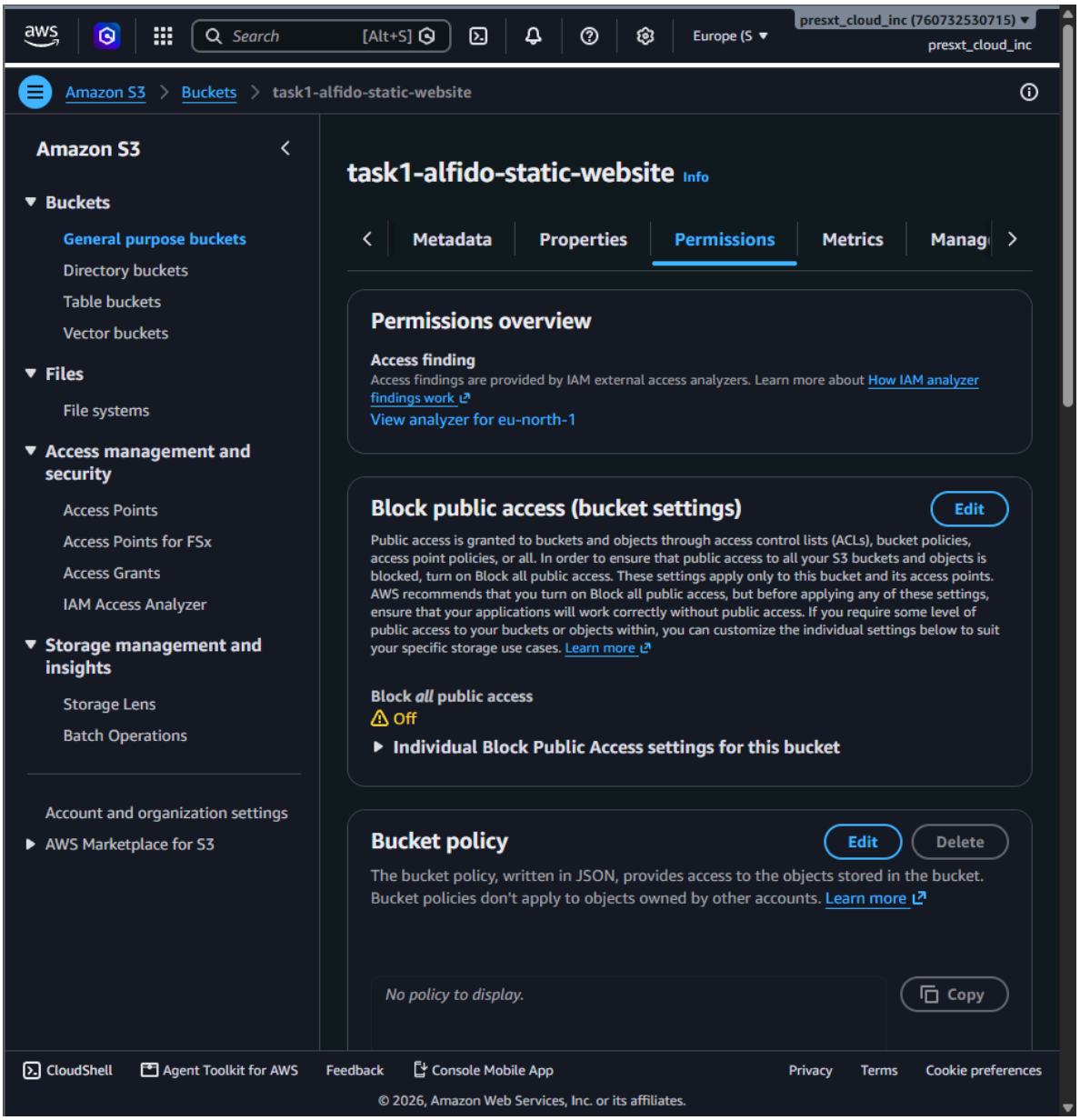

Fig 4.8.1(g): Bucket permissions overview with Block Public Access turned off

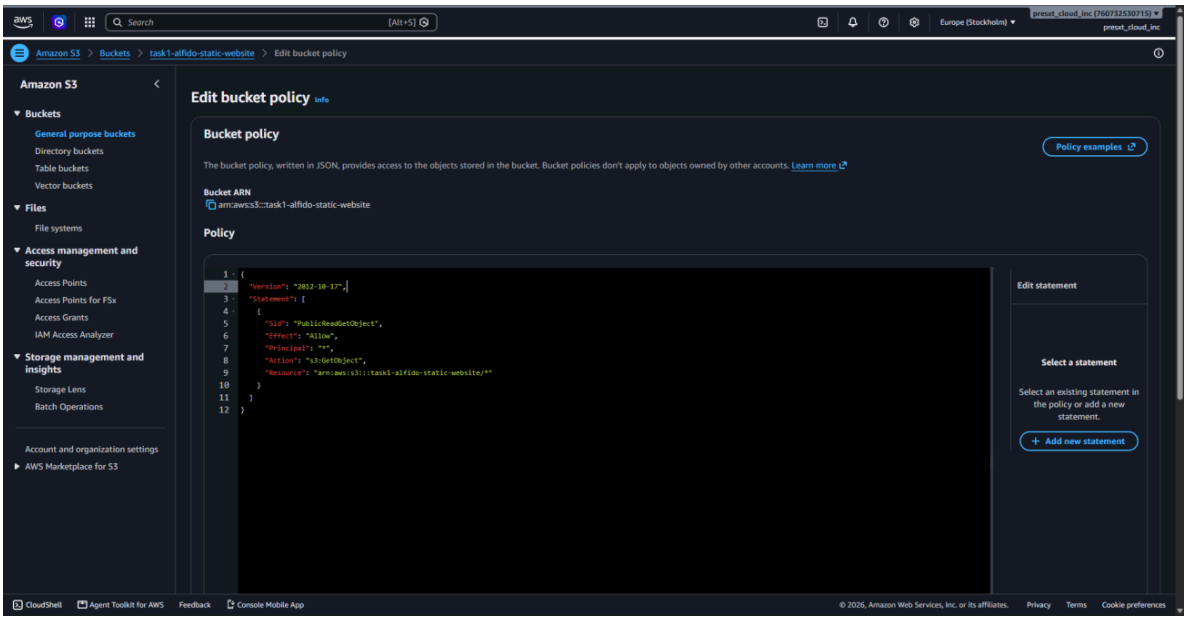

Fig 4.8.1(h): S3 bucket policy granting public s3:GetObject access

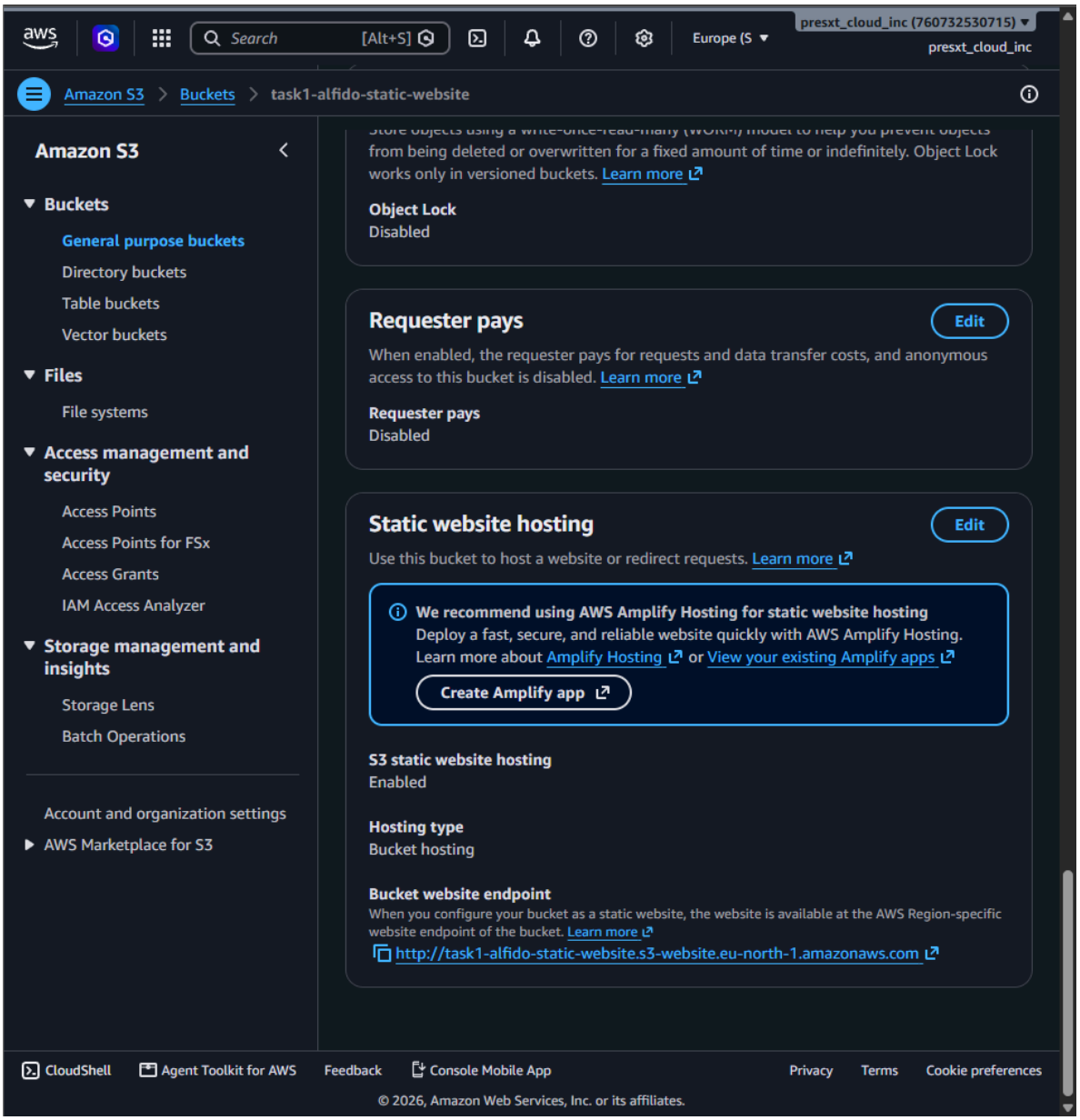

Fig 4.8.1(i): Static website hosting enabled, showing the bucket website endpoint

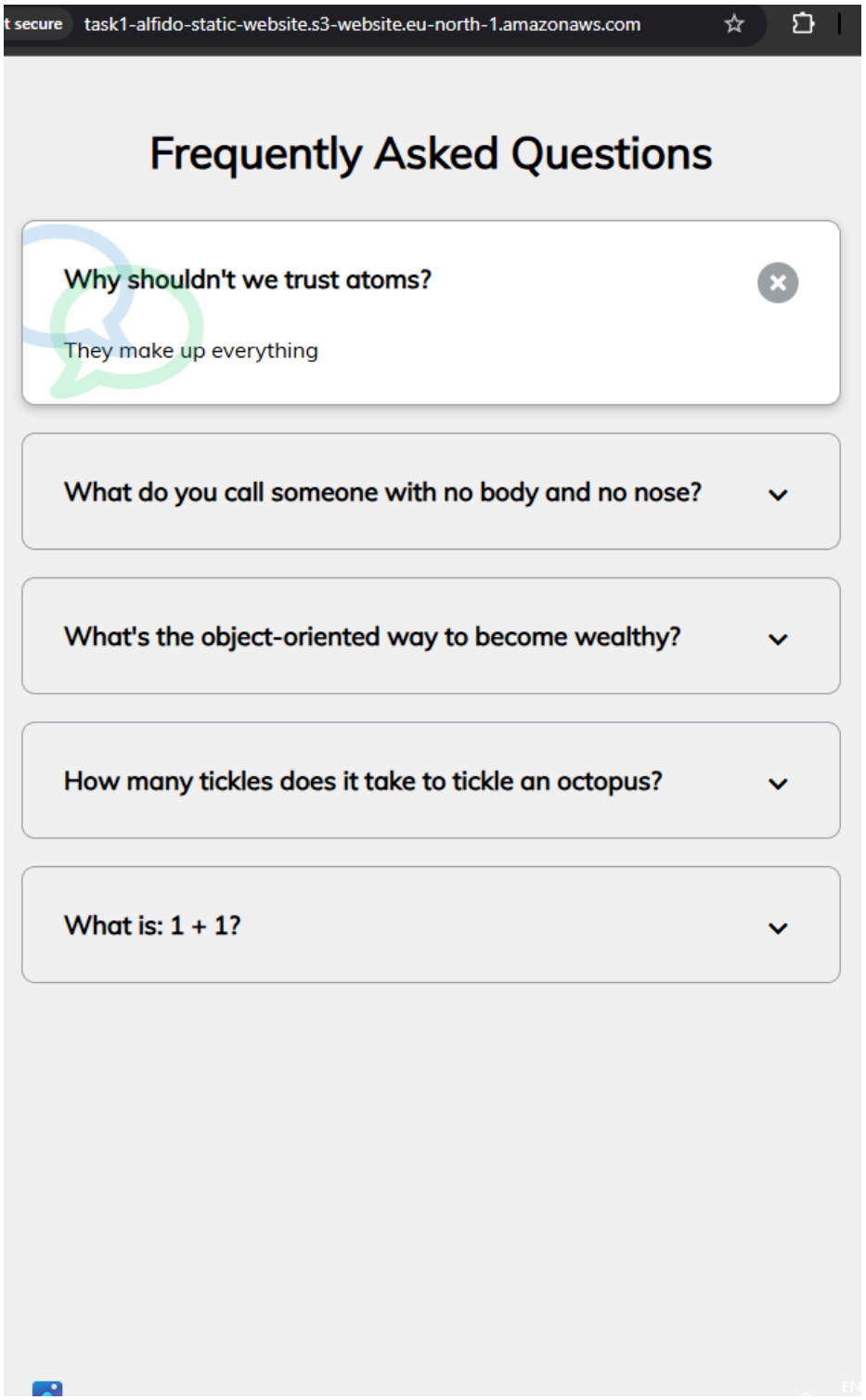

Fig 4.8.1(j): Live static website accessed publicly via the S3 website endpoint

Result: The static website was successfully deployed and accessed through the public S3 website endpoint.

4.8.2 Task 2: Deploy a Virtual Machine & Install Web Server

Objective: To create an Ubuntu virtual machine on AWS EC2, configure port 80, install the Apache web server, and host a custom “Hello from Cloud VM” webpage.

Tools Used: AWS EC2, Ubuntu Server, Apache2.

Cloud Provider: AWS EC2

Operating System: Ubuntu Server

Instance Name: task2-alfido-cloud-vm

Public IPv4: 13.60.163.6

Web Server: Apache2

Port: 80

Procedure:

- Created an Ubuntu Server virtual machine using AWS EC2.
- Configured the security group to allow HTTP traffic on port 80 (and SSH on port 22).
- Connected to the VM using EC2 Instance Connect.
- Updated the Ubuntu package repository.
- Installed the Apache2 web server.
- Started and enabled the Apache service.
- Created a custom index.html page displaying “Hello from Cloud VM”.
- Accessed the webpage using the VM public IPv4 address.

Commands Used:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
echo '<h1>Hello from Cloud VM</h1><p>Alfido Tech Internship - Task 2</p>' | sudo tee /var/www/html/index.html
```

Screenshots / Evidence:

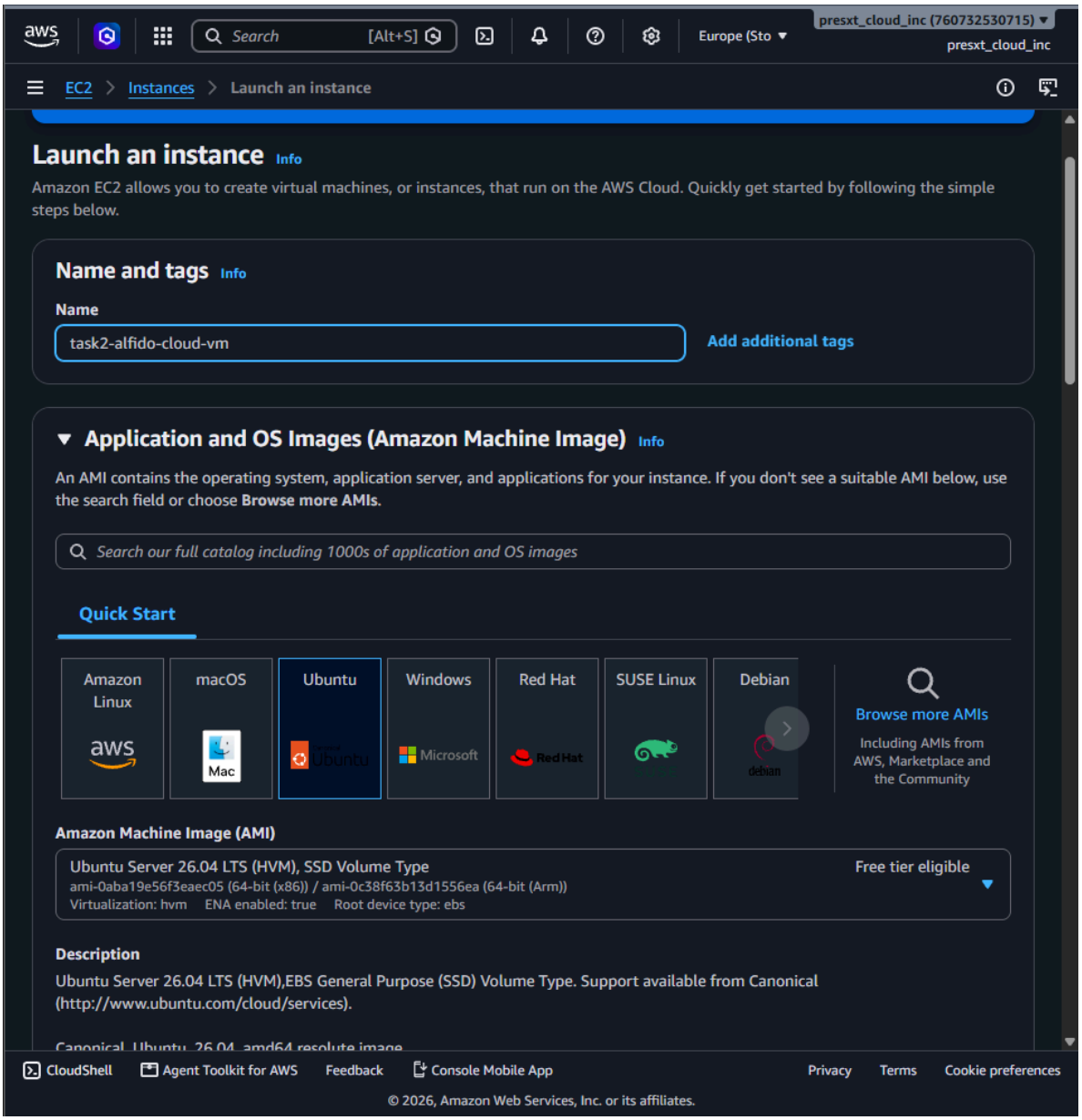

Fig 4.8.2(a): Launching an EC2 instance named "task2-alfido-cloud-vm" with Ubuntu Server AMI

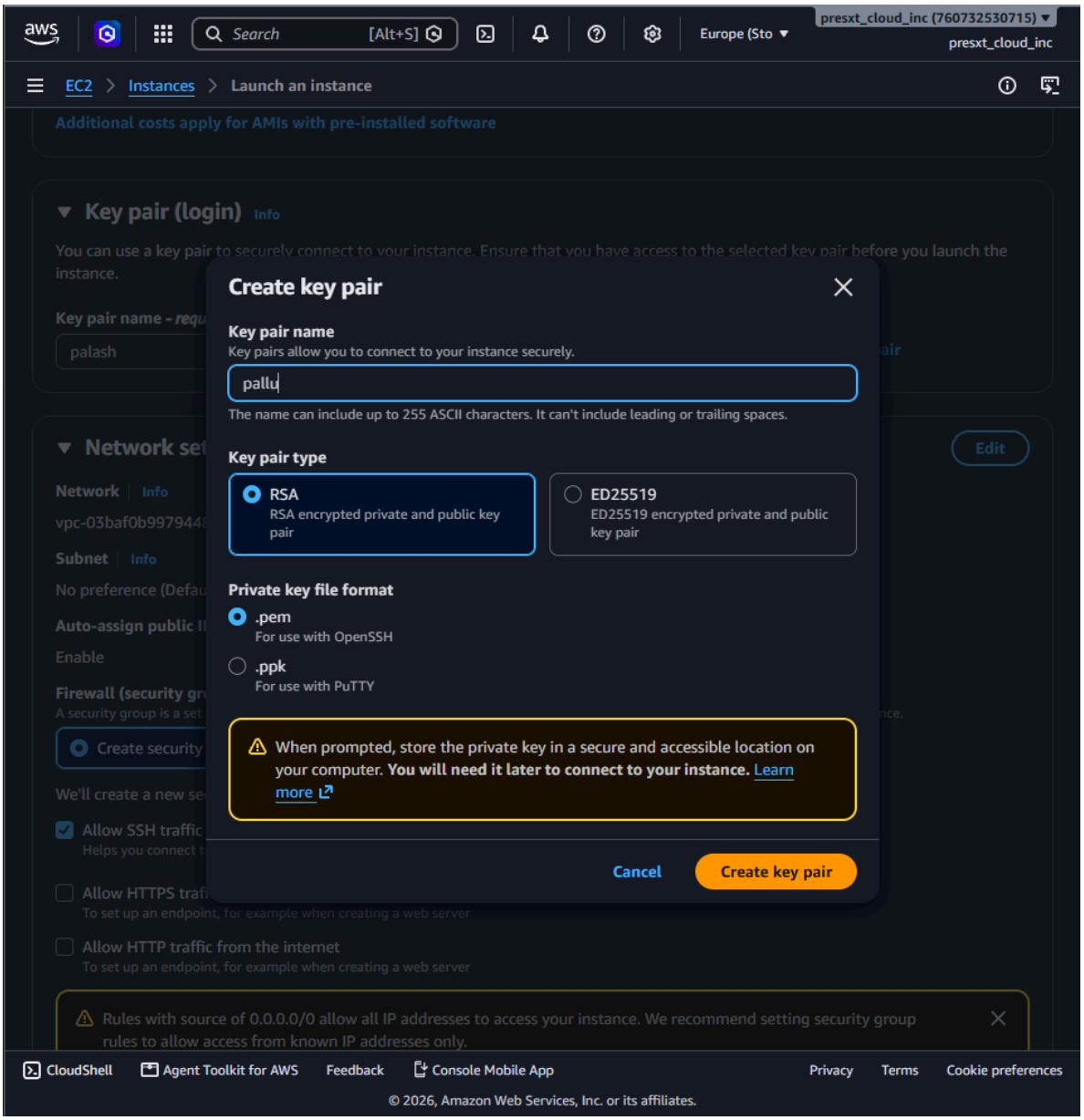

Fig 4.8.2(b): Creating a key pair for secure SSH access to the instance

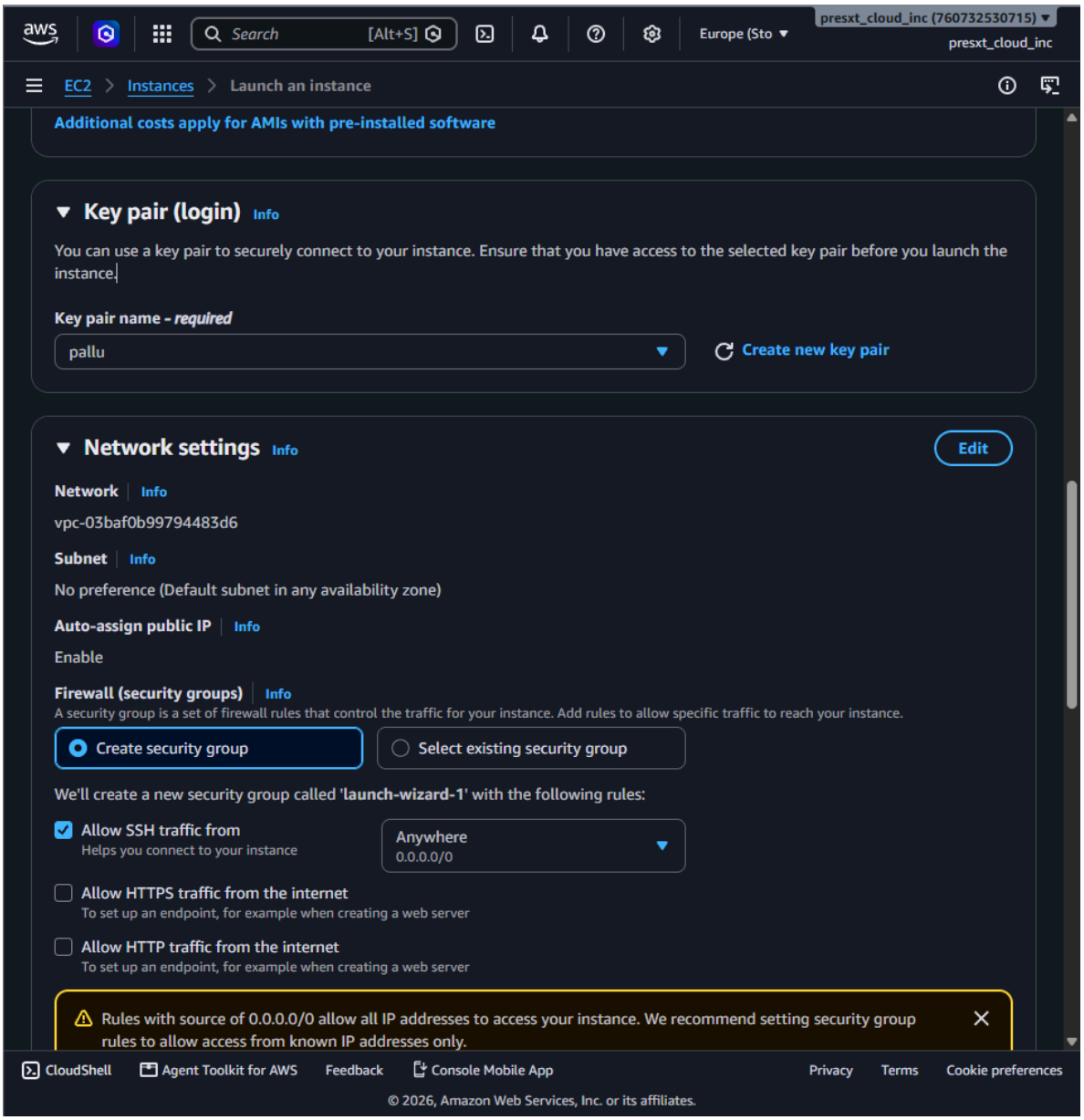

Fig 4.8.2(c): Reviewing key pair and network settings before launch

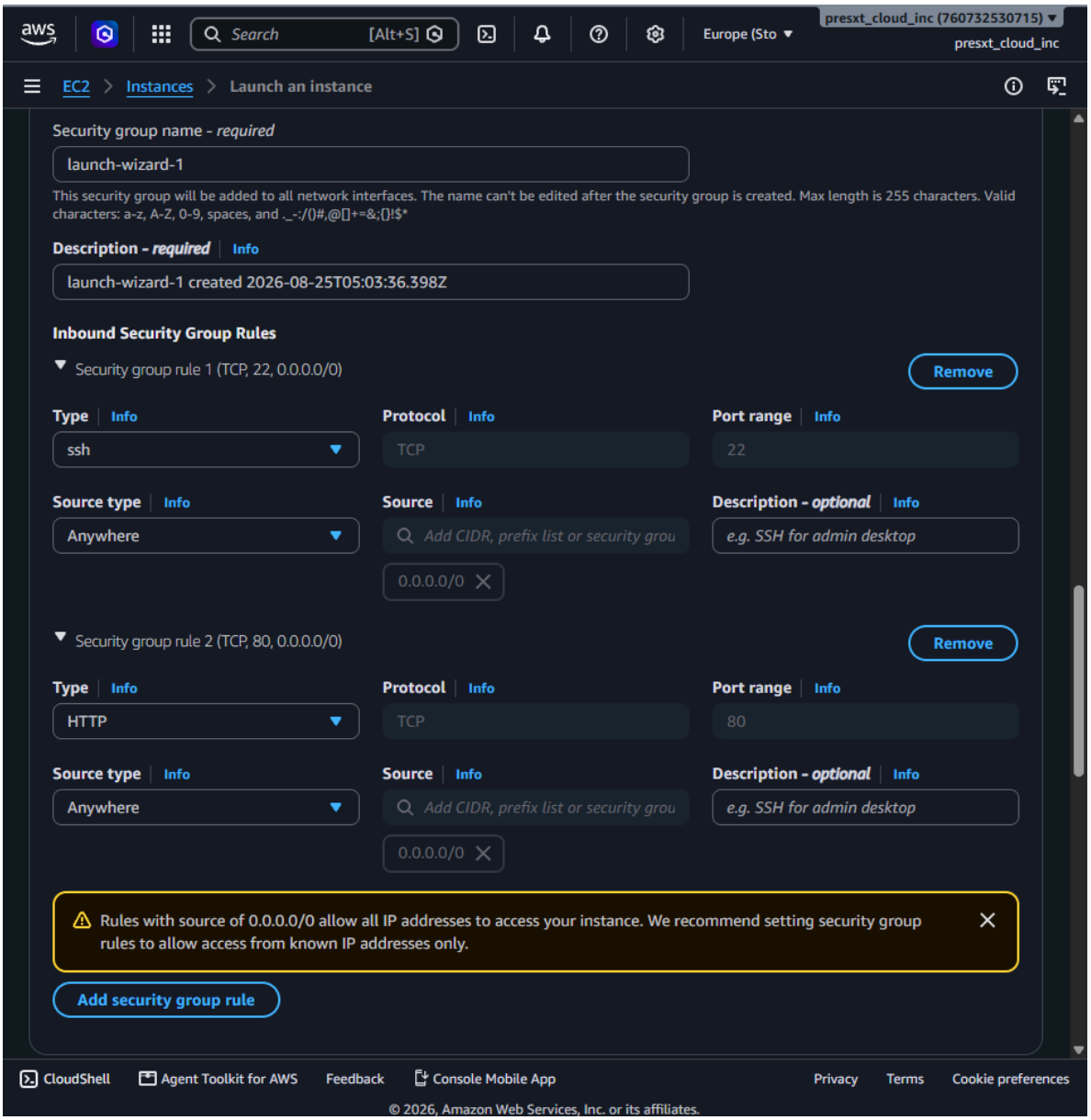

Fig 4.8.2(d): Configuring the security group to allow SSH (port 22) and HTTP (port 80) traffic

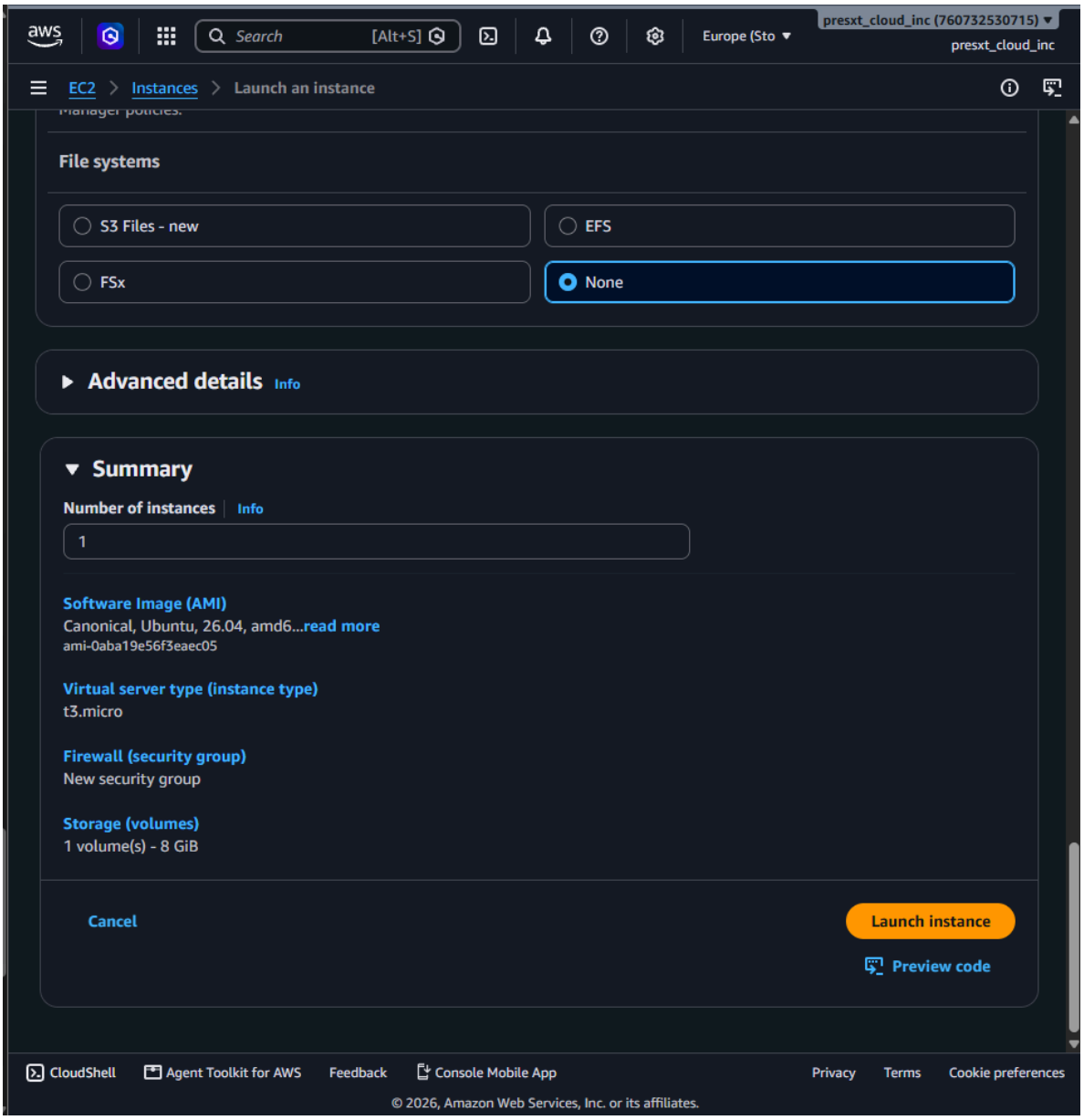

Fig 4.8.2(e): Instance launch summary (AMI, instance type, security group, storage)

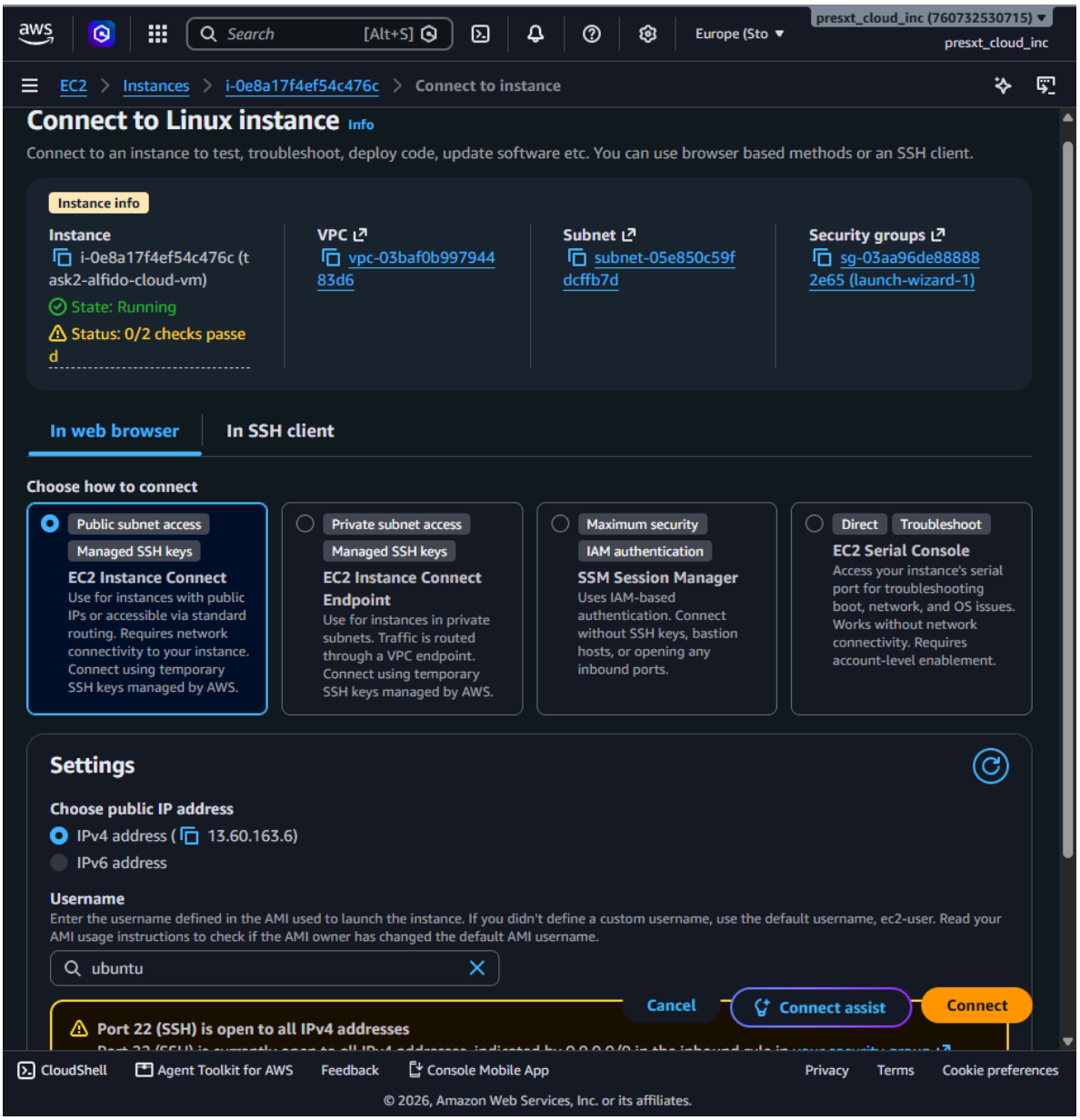

Fig 4.8.2(f): Connecting to the instance using EC2 Instance Connect

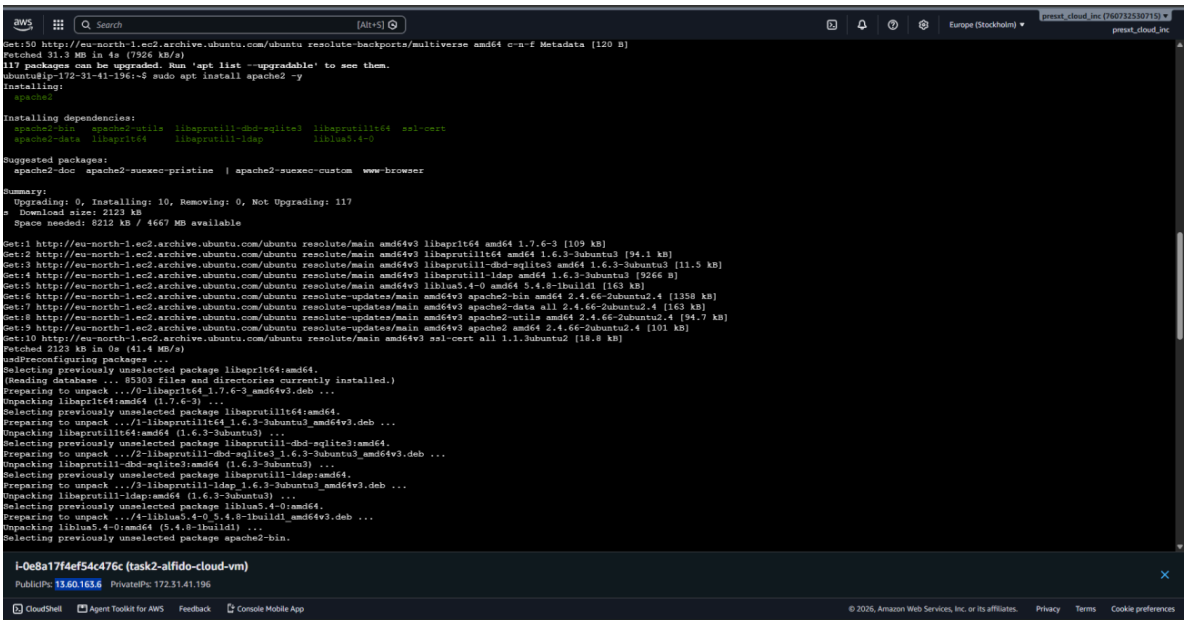

Fig 4.8.2(g): Terminal output of package update and Apache2 installation

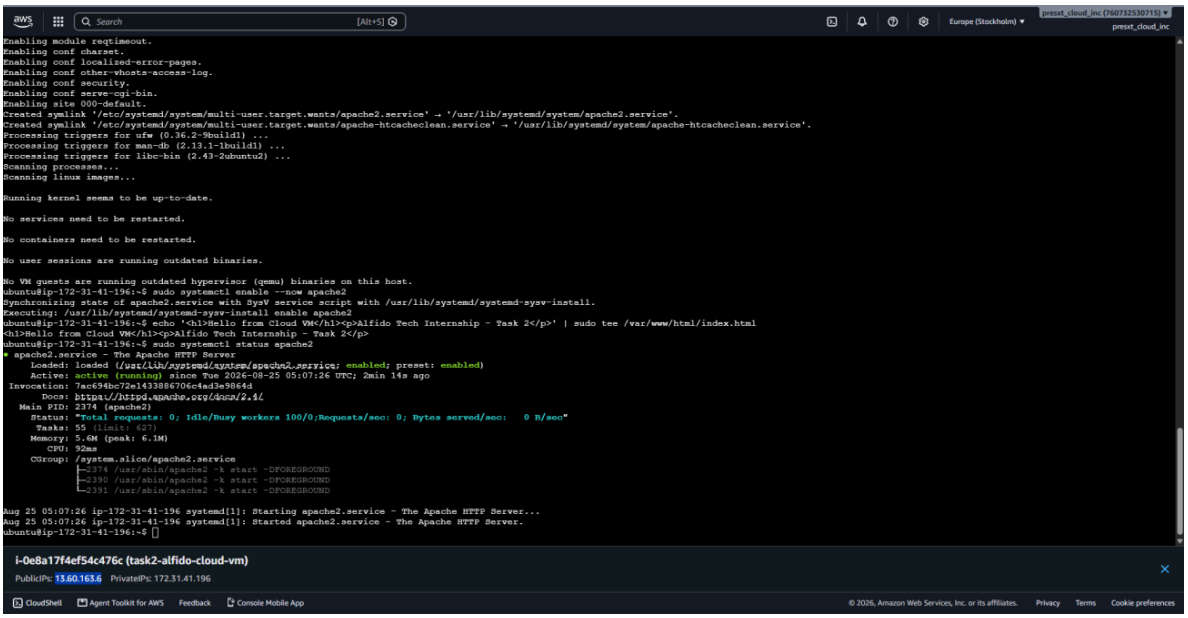

Fig 4.8.2(h): Enabling and starting the Apache2 service, and verifying its running status

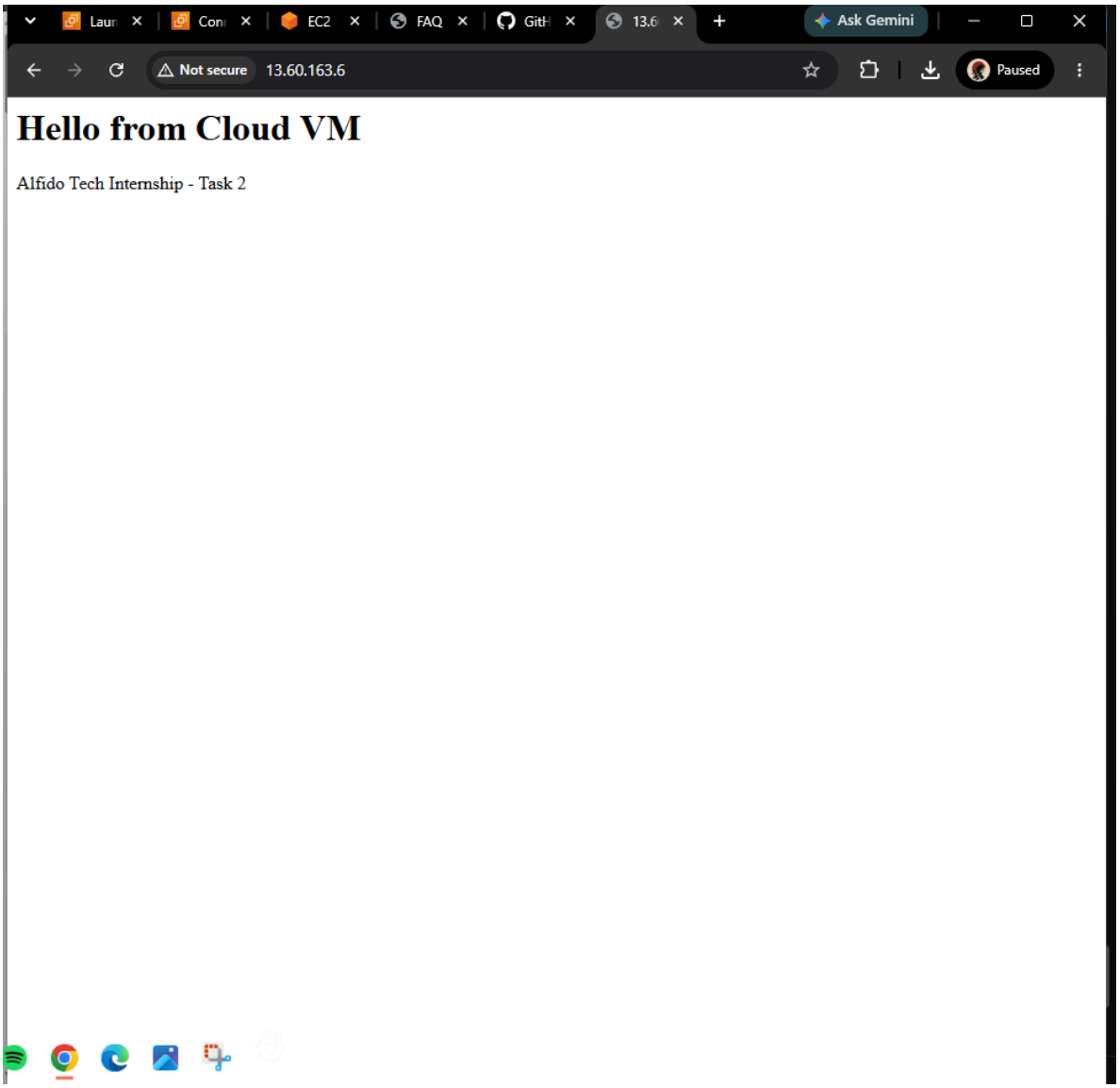

Fig 4.8.2(i): Custom "Hello from Cloud VM" webpage accessed via the instance public IPv4 address

Live Page URL: http://13.60.163.6

Result: The Ubuntu cloud VM was successfully deployed, the Apache web server was installed and configured, and the custom webpage was successfully accessed through the VM public IP address.

4.8.3 Task 3: Create a Serverless Function

Objective: To create a simple serverless function that returns a JSON output.

Tools Used: AWS Lambda, Python, Amazon API Gateway.

Function Name: task3-alfido-serverless

Endpoint URL: https://5vf3sv561b.execute-api.eu-north-1.amazonaws.com/default/task3-alfido-serverless

Procedure:

- Created an AWS Lambda function using Python.
- Configured the Lambda function to return a JSON response.
- Tested the function using the AWS Lambda cloud console.
- Verified the successful execution and JSON output.
- Created an API Gateway trigger for the Lambda function.
- Configured the API Gateway endpoint for public (open) access.
- Deployed the function and obtained the public API endpoint URL.
- Opened the endpoint in a browser and verified that it returned JSON output.

Lambda Function Code:

```
import json

def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Hello from Alfido Tech",
            "task": "Task 3",
            "status": "success"
        })
    }
```

Screenshots / Evidence:

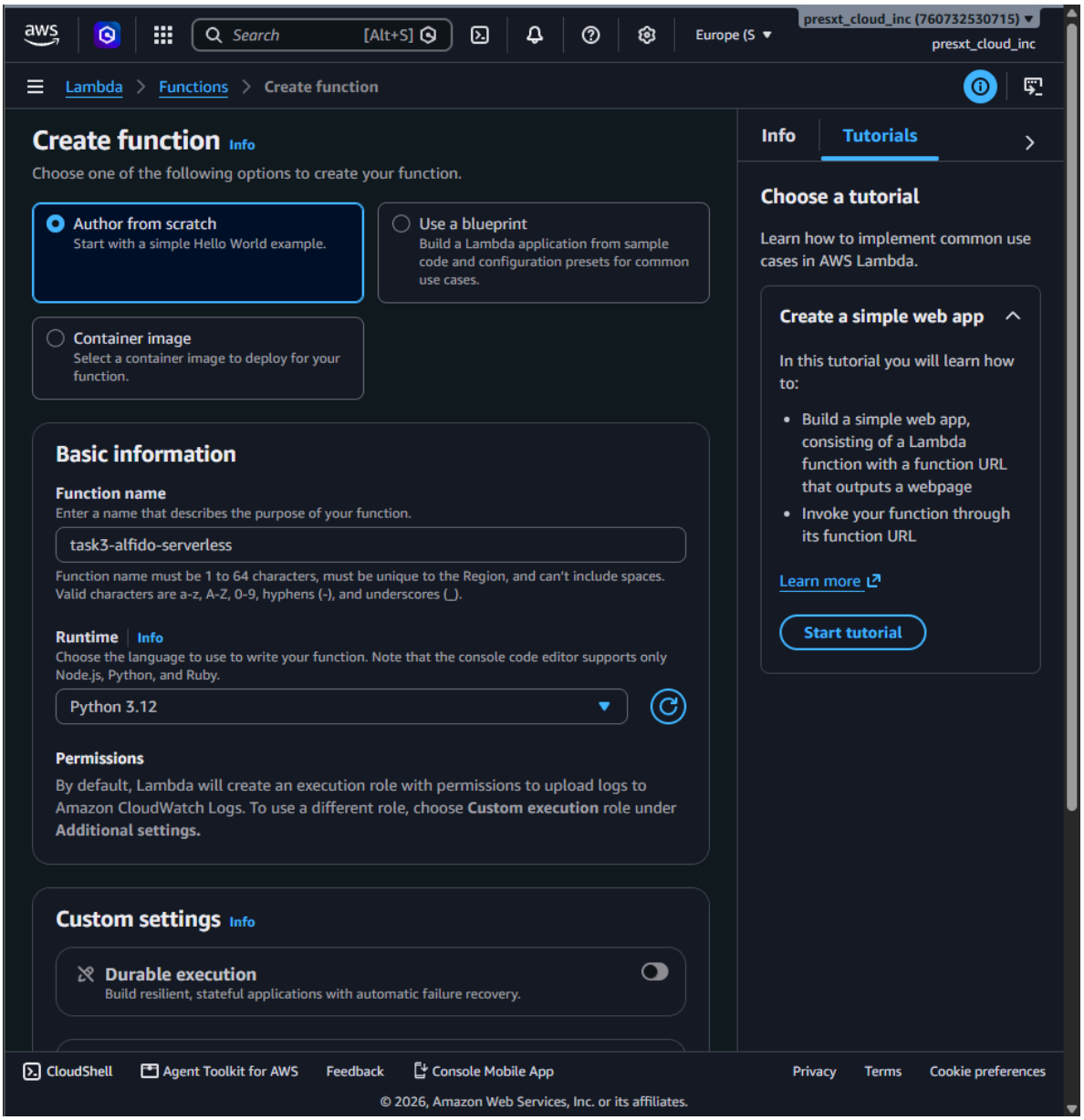

Fig 4.8.3(a): Creating the Lambda function "task3-alfido-serverless" using the Python 3.12 runtime

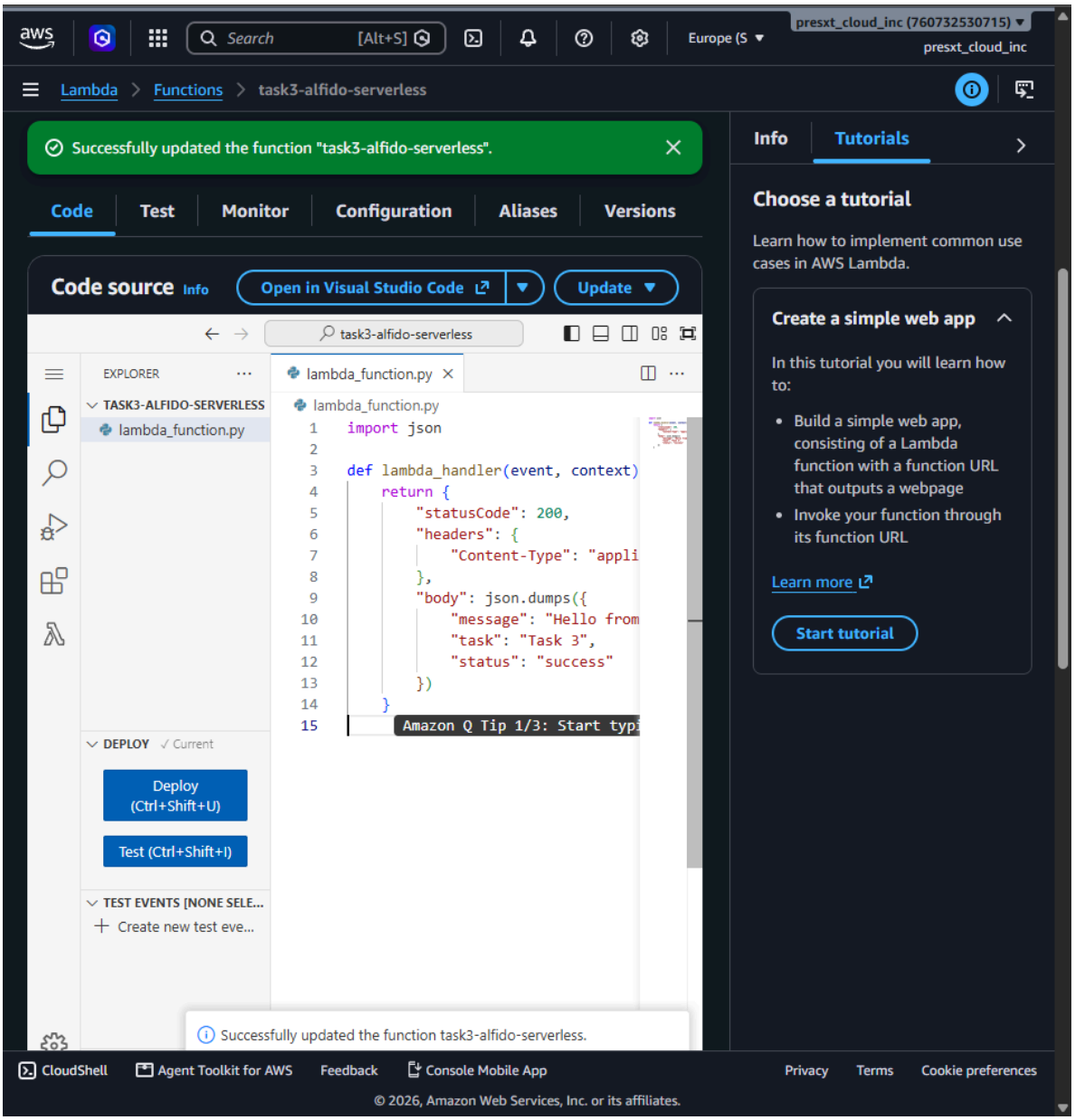

Fig 4.8.3(b): Lambda function code (lambda_function.py) after successful deployment

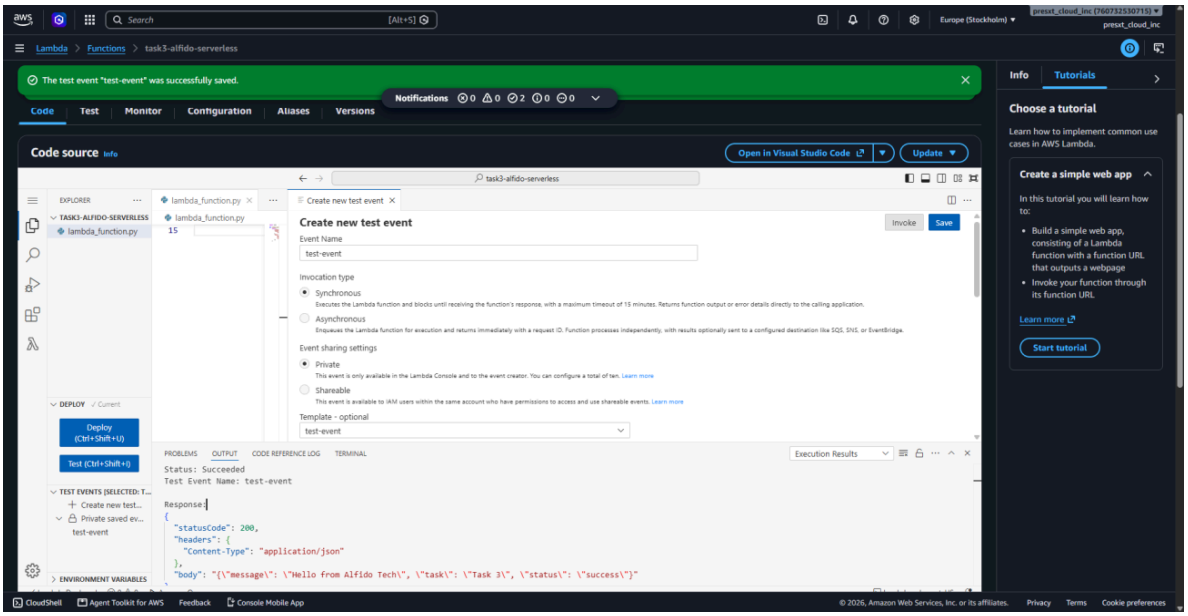

Fig 4.8.3(c): Creating and invoking a test event, with a successful JSON execution result

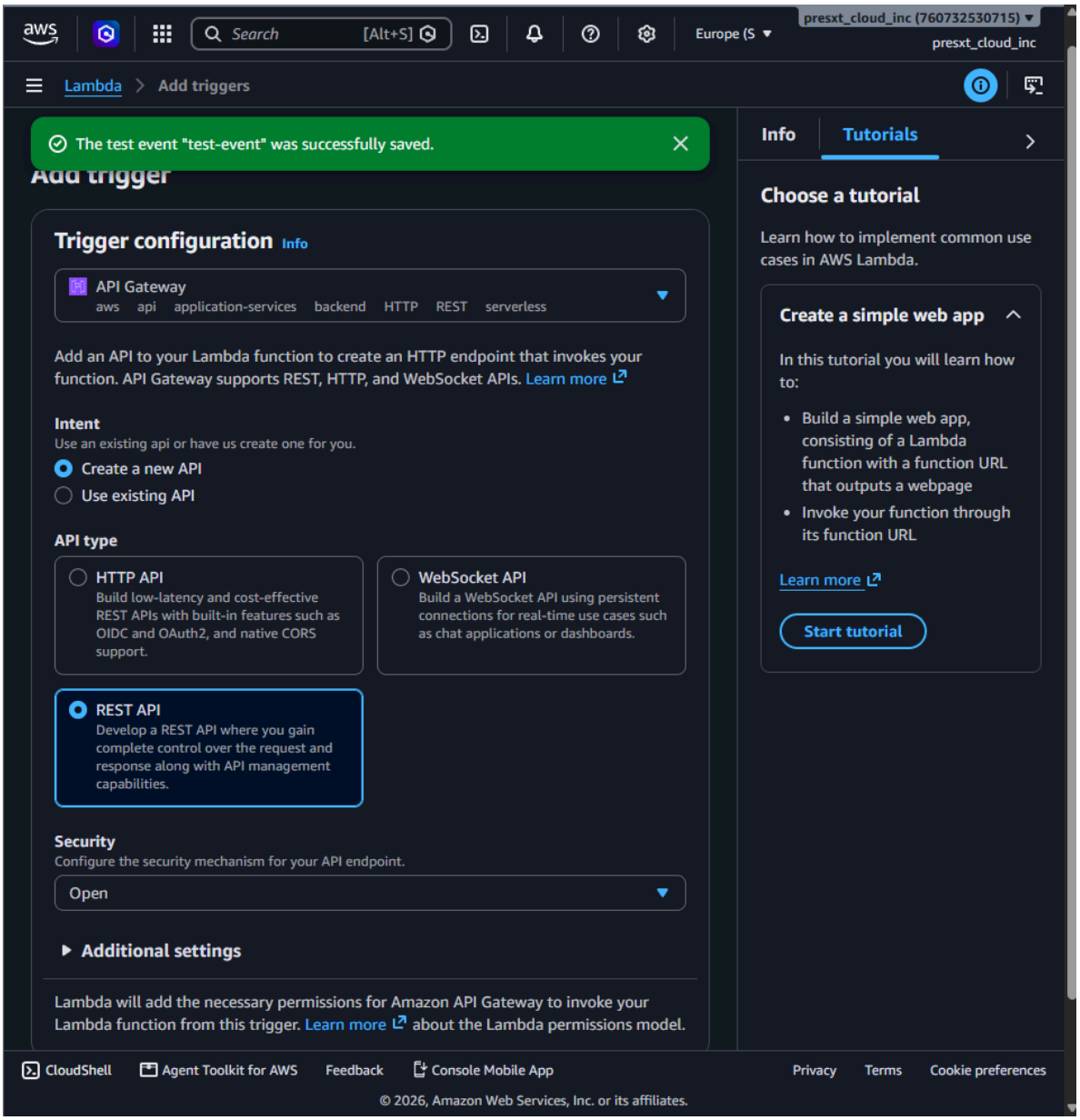

Fig 4.8.3(d): Configuring an API Gateway (REST API) trigger for the Lambda function

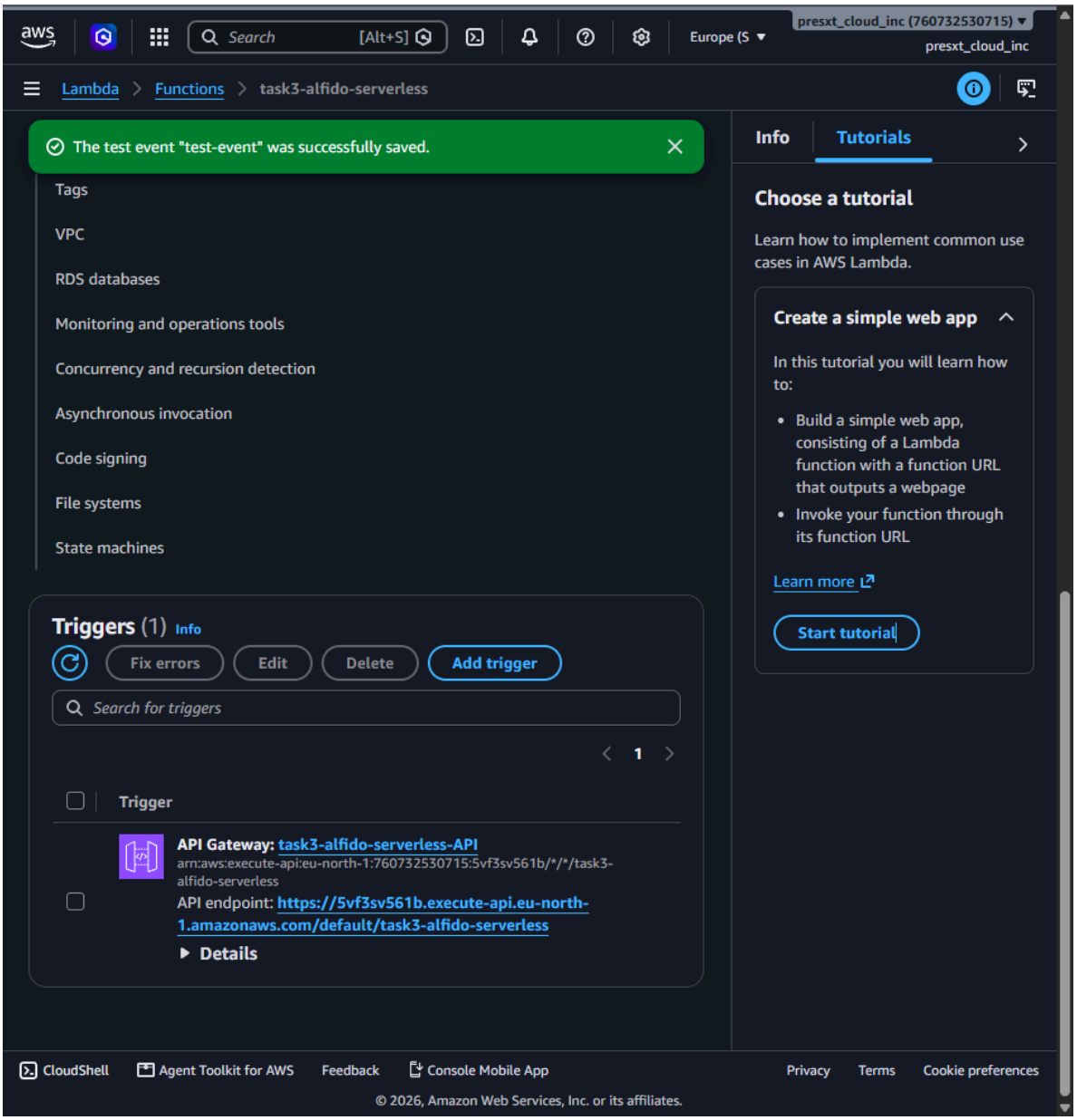

Fig 4.8.3(e): API Gateway trigger details showing the generated public API endpoint

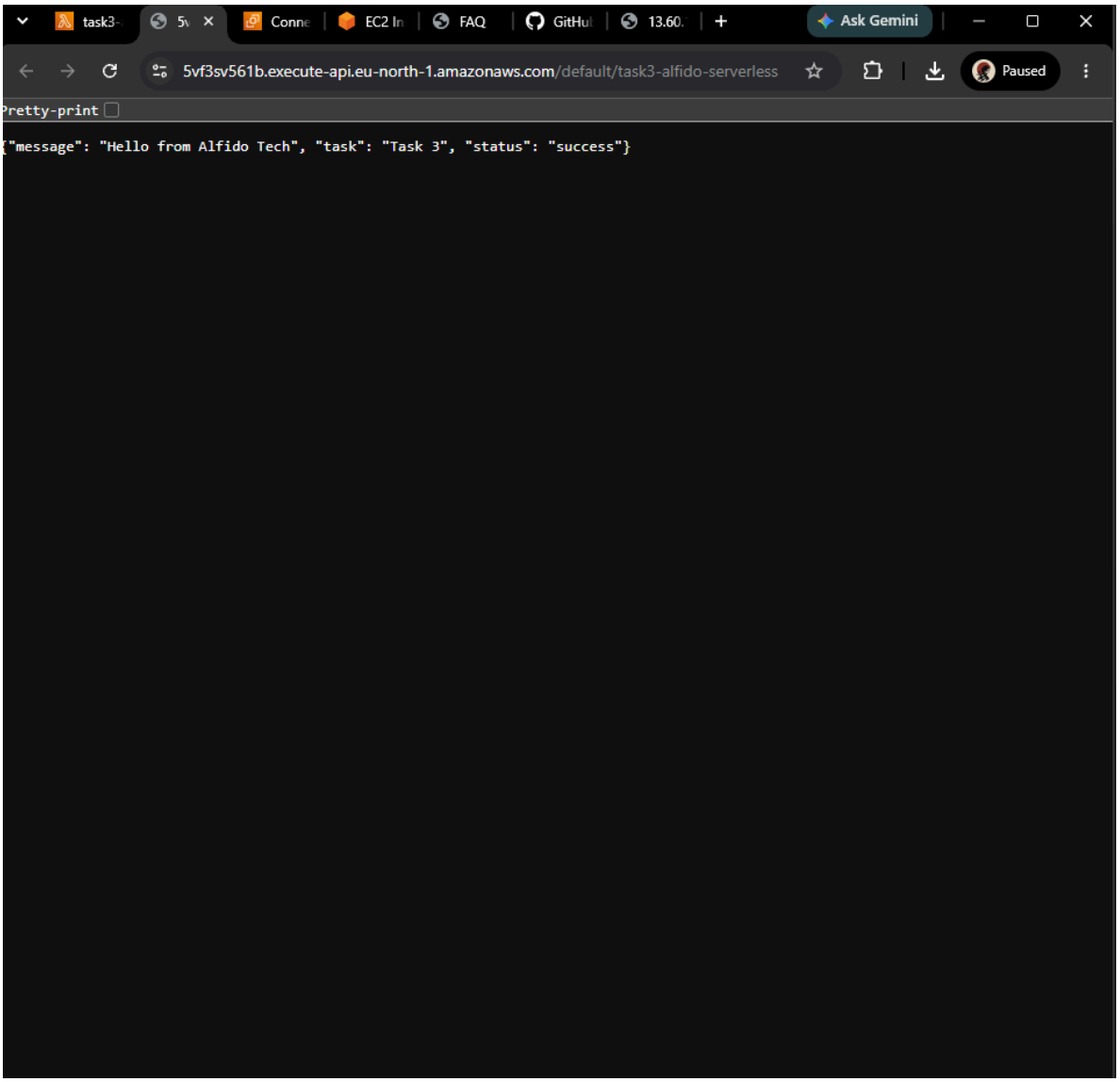

Fig 4.8.3(f): Public API endpoint opened in a browser, returning the expected JSON response

Result: The serverless function was successfully created, tested, deployed, and exposed through a public API Gateway endpoint that returns JSON output.

4.8.4 Task 4: Deploy a Docker Container on Cloud VM

Objective: To deploy a Docker container on an AWS cloud virtual machine and make the containerized web application publicly accessible.

Tools Used: AWS EC2, Ubuntu Server, Docker, Nginx.

VM Name: task4-alfido-docker

Operating System: Ubuntu Server

Container Image: Nginx

Container Name: alfido-nginx

Port: 80

Public IP: 13.60.81.133

Website URL: http://13.60.81.133/

Procedure:

- Created a separate Ubuntu Server virtual machine using AWS EC2.
- Configured the security group to allow SSH traffic on port 22 and HTTP traffic on port 80.
- Connected to the cloud VM using EC2 Instance Connect.
- Updated the Ubuntu package repository.
- Installed Docker on the virtual machine.
- Enabled and started the Docker service.
- Verified the Docker installation.
- Pulled and ran an Nginx Docker container.
- Mapped the container port 80 to the VM port 80.
- Verified that the Nginx container was running using docker ps.
- Accessed the containerized webpage using the VM public IPv4 address.
- Verified that the “Welcome to nginx!” webpage was publicly accessible.

Docker Commands Used:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo docker --version
```

sudo docker run -d --name alfido-nginx -p 80:80 nginx
sudo docker ps

Screenshots / Evidence:

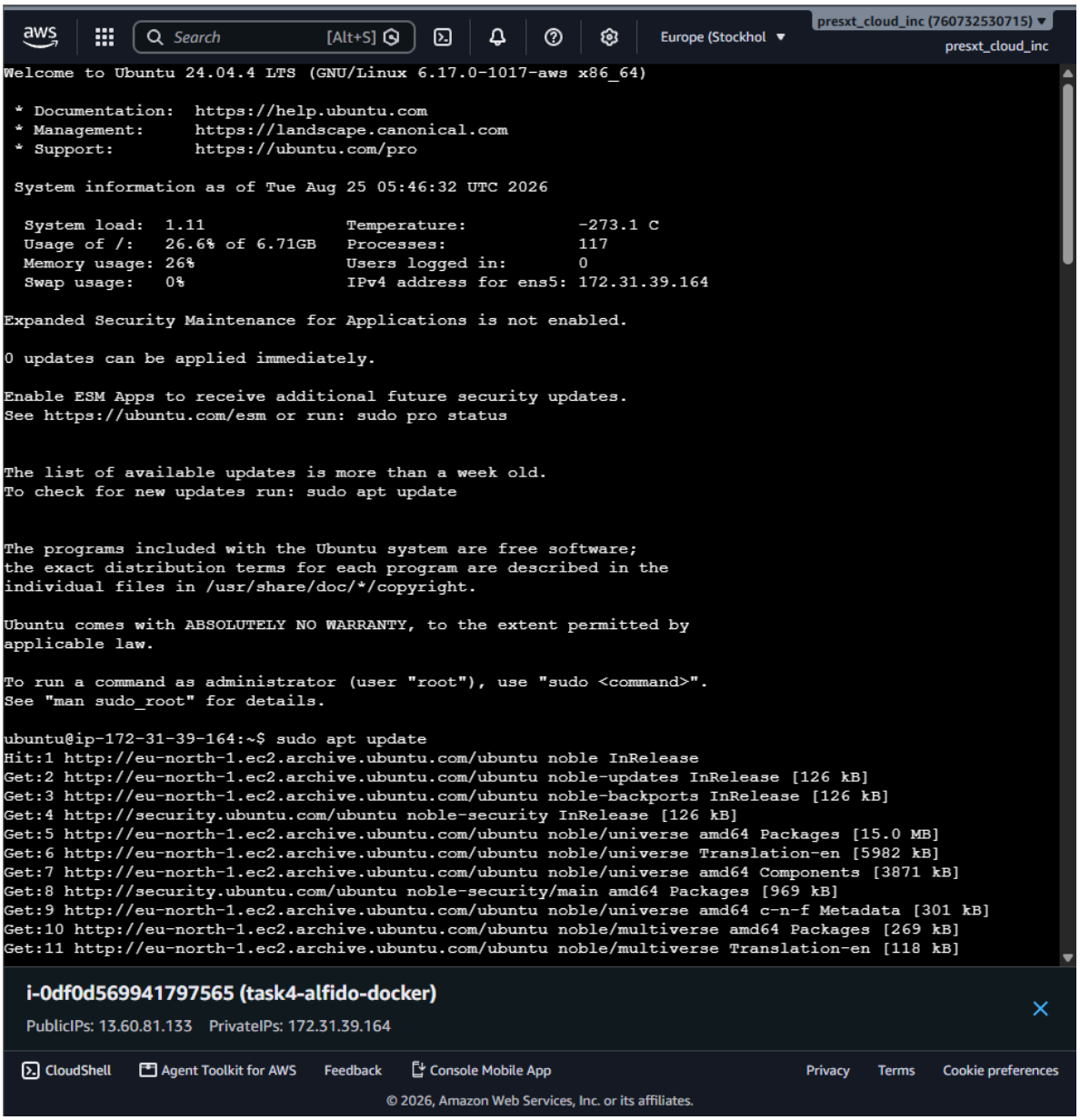

Fig 4.8.4(a): Connecting to the "task4-alfido-docker" EC2 instance and updating the Ubuntu package list

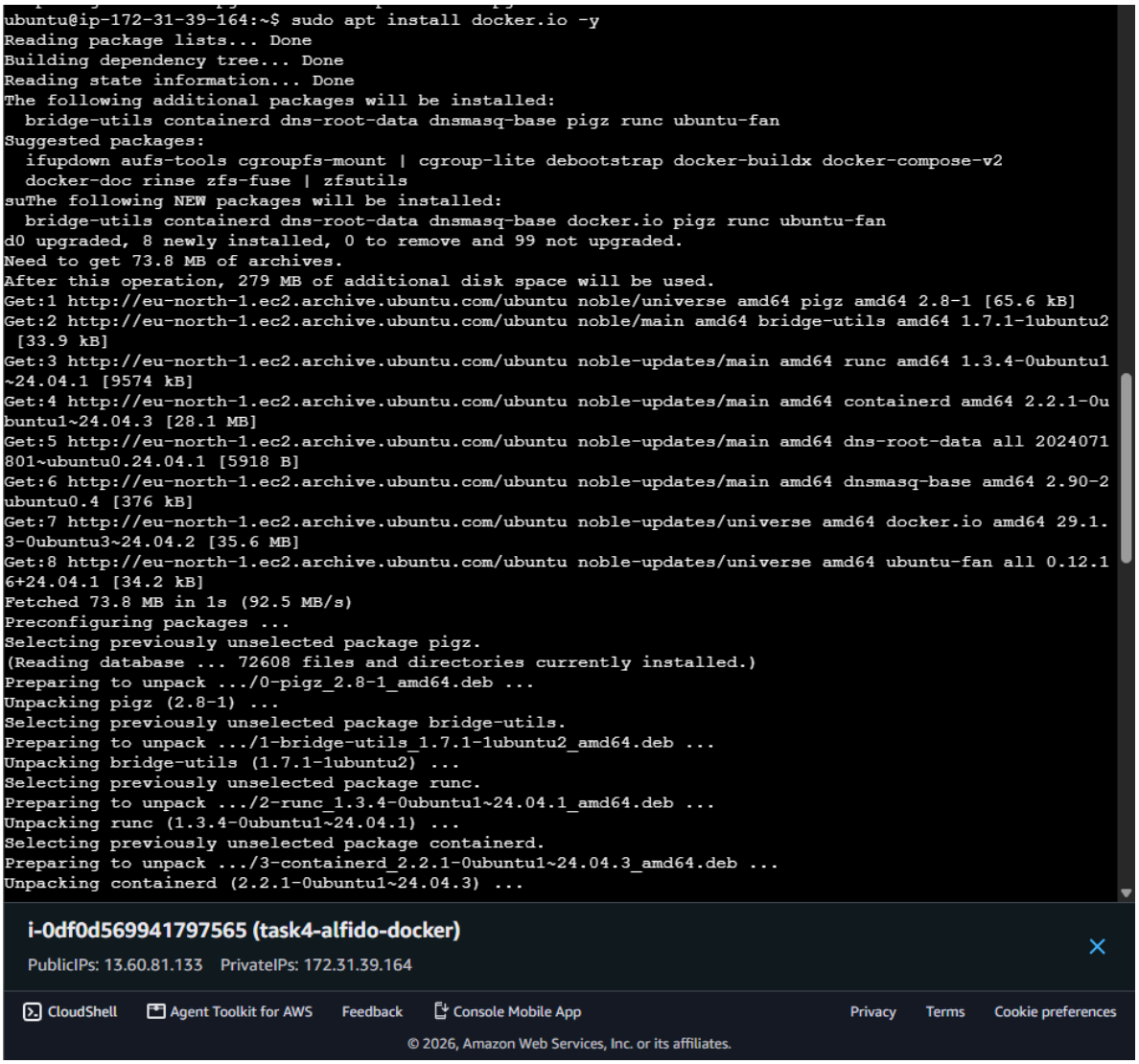

Fig 4.8.4(b): Installing Docker (docker.io) on the Ubuntu Server virtual machine

Result: The Docker container was successfully deployed on an AWS EC2 cloud VM. The Nginx container was running successfully, and its webpage was accessed publicly through the VM public IP address.

4.9 Expected Outcomes

- A publicly accessible static website hosted entirely on Amazon S3, without the need for a traditional web server.
- A cloud-hosted Ubuntu virtual machine running Apache2, serving a custom webpage over HTTP on port 80.
- A working serverless function on AWS Lambda that can be invoked over the internet through an API Gateway endpoint and returns a structured JSON response.
- A Docker container (Nginx) running on a cloud virtual machine, with its web page accessible publicly through the VM IP address.
- Practical, hands-on familiarity with core AWS services (S3, EC2, Lambda, API Gateway), Linux server administration, and containerization using Docker.

4.10 Certificates of Completion and Other Communication/Mail Proof

The following evidence is attached as part of the internship documentation.

ALFIDO TECH

RIDING THE WAVES OF TECHNOLOGICAL ADVANCEMENT

Hyderabad, Telangana, India
internship@alfidotech.com
www.alfidotech.com
Date: 10/08/2026

Internship Offer Letter

Dear Palash Malviya,

Candidate ID: BS/REG/126233

We are delighted to offer you the position of **Cloud Computing Intern** at **Alfido Tech**. Your academic background, skills, and overall enthusiasm clearly reflect your potential, and we believe that you will be an excellent addition to our internship program.

At Alfido Tech, we are committed to nurturing emerging talent by providing a structured, growth-oriented learning environment. Throughout your internship, you will engage in meaningful, hands-on assignments designed to develop your technical competence, problem-solving abilities, and professional confidence.

This internship follows a **task-based learning model**, enabling you to gain practical exposure through real-world projects relevant to your chosen domain. Each task has been designed to simulate actual industry work, helping you build a strong technical foundation and preparing you for future professional opportunities.

You will also receive access to curated learning resources, timely guidance from mentors, and support from our internship coordination team. We encourage you to maintain active communication, seek clarification whenever required, and approach each task with dedication and a willingness to learn.

Upon successful completion of all assigned tasks and adherence to internship guidelines, you will be awarded an official **Internship Completion Certificate**. Based on your performance and overall contribution, you may also become eligible for a **Letter of Recommendation (LOR)**, which can serve as a valuable credential in your future academic or professional pursuits.

During your internship journey, we expect you to demonstrate professionalism, discipline, and a positive learning attitude. We believe that your contributions will not only support your individual growth but will also align with Alfido Tech's vision of fostering innovation and excellence.

We are excited to welcome you to **Alfido Tech** and look forward to seeing you develop your skills, achieve your goals, and make the most of this opportunity.

We wish you great success ahead. Let's grow together!

Duration: 1 Month
Starting Date: 10/08/2026

Sincerely,

Founder & CEO

AICTE

ISO 9001:2015

Fig 4.10.1: Internship Offer Letter issued by Alfido Tech (Candidate ID: BS/REG/126233, dated 10/08/2026)

5. BIBLIOGRAPHY / REFERENCES

[1] Amazon Web Services, “Amazon S3 - Static website hosting,” AWS Documentation.

[2] Amazon Web Services, “Amazon EC2 User Guide,” AWS Documentation.

[3] Amazon Web Services, “AWS Lambda Developer Guide,” AWS Documentation.

[4] Amazon Web Services, “Amazon API Gateway Developer Guide,” AWS Documentation.

[5] Docker Inc., “Docker Documentation.”

[6] The Apache Software Foundation, “Apache HTTP Server Documentation.”

[7] Canonical Ltd., “Ubuntu Server Documentation.”

[8] Alfido Tech, Cloud Computing Internship - Task 1 to Task 4 completion documents (internal internship material), 2026.